MLFP mlfp02 — Textbook Chapter

- Module 2 — Statistical Mastery for Machine Learning and AI Success
  - Learning Outcomes
  - Prerequisites
  - How to Read This Chapter
- Lesson 2.1 — Probability and Bayesian Thinking
  - Why This Matters
  - Core Concepts — Foundations
    - What is probability?
    - Independent vs. dependent events
    - Conditional probability
    - Base rate fallacy (ART test example)
  - Mathematical Foundations — Theory
    - Deriving Bayes' theorem
    - Expected value
    - Variance — two equivalent forms
    - Numerical example — HDB prices
    - Distributions — the statistics theme park
    - Sampling bias — the friendship paradox
  - The Kailash Engine — ExperimentTracker
  - Worked Example — Full Bayesian Update on HDB Prices
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 2.2 — Parameter Estimation and Inference
  - Why This Matters
  - Core Concepts — Foundations
    - Population vs sample
    - Sampling distribution
    - Law of Large Numbers (LLN)
    - Central Limit Theorem (CLT)
    - Confidence intervals — what they really mean
    - Bessel's correction — why `n - 1`
  - Mathematical Foundations — MLE from First Principles
    - The idea in one sentence
    - Derivation 1 — MLE for the Normal distribution
    - Derivation 2 — MLE for the Binomial / Bernoulli
    - Fisher information (THEORY)
    - MAP — adding a prior (THEORY)
    - When MLE fails (ADVANCED)
  - The Kailash Engine — ExperimentTracker + ModelVisualizer
  - Worked Example — MLE, Fisher Information, and a Confidence Interval
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 2.3 — Bootstrapping and Hypothesis Testing
  - Why This Matters
  - Core Concepts — The Bootstrap
    - Efron's insight
    - Why "with replacement"?
    - Percentile confidence interval

        - BCa intervals (THEORY)
        - When the bootstrap fails
        - Parametric vs non-parametric bootstrap
    - Mathematical Foundations — Hypothesis Testing
        - The Neyman-Pearson framework
        - p-values — what they really mean
        - Type I vs Type II errors
        - The t-statistic
        - Two-sample t-test
        - Permutation tests
        - Multiple testing
    - The Kailash Engine — ExperimentTracker for A/B Tests
    - Worked Example — Full A/B Analysis
    - Try It Yourself
    - Cross-References
    - Reflection
- Lesson 2.4 — A/B Testing and Experiment Design
    - Why This Matters
    - Core Concepts — Experiment Design
        - Hypothesis formulation
        - Randomisation
        - The data collection framework — why / what / where / how / frequency
        - Common data collection problems
        - Clean DataOps architecture
        - Sample Ratio Mismatch (SRM)
    - The Kailash Engine — ExperimentTracker for Design
    - Worked Example — Designing the BOGO Banner Test
    - Try It Yourself
    - Cross-References
    - Reflection
- Lesson 2.5 — Linear Regression
    - Why This Matters
    - Core Concepts
        - The linear model
        - Ordinary Least Squares
        - OLS as MLE under Normal errors
        - Interpreting coefficients
        - Non-linear extensions in a linear framework
        - Categorical encoding
    - Mathematical Foundations — Inference
        - Standard error of a coefficient
        - t-statistic
        - R-squared
        - F-statistic
        - Numerical example — HDB with two predictors
    - The Kailash Engine — TrainingPipeline + ModelVisualizer
    - Worked Example — Predicting HDB Prices
    - Try It Yourself
    - Cross-References
    - Reflection
- Lesson 2.6 — Logistic Regression and Classification Foundations
    - Why This Matters
    - Core Concepts — From Linear to Logistic
        - The linearity problem for binary responses

        - Logistic regression model
        - Odds ratio interpretation
    - Mathematical Foundations — Maximum Likelihood for Logistic Regression
        - The likelihood
        - Gradient and why there's no closed form
        - Sigmoid derivative (useful identity)
        - Multiclass extensions
        - Evaluation preview
    - Core Concepts — ANOVA (one-way)
        - Why ANOVA?
        - The F-statistic decomposition
        - Numerical example — HDB across flat types
        - Post-hoc tests
        - ANOVA vs regression
    - The Kailash Engine — TrainingPipeline for Classification
    - Worked Example — Employee Attrition
    - Try It Yourself
    - Cross-References
    - Reflection
- Lesson 2.7 — CUPED and Causal Inference
    - Why This Matters
    - Core Concepts — CUPED
        - The idea
        - The adjustment
        - Deriving the optimal θ
        - Deriving the variance reduction
        - Sample size multiplier
        - Caveats
    - Core Concepts — Causal Inference Basics
        - Rubin's potential outcomes
    - Mathematical Foundations — Difference-in-Differences
        - The setup
        - The parallel trends assumption
        - Regression form
        - Placebo tests
        - Numerical example — Singapore ABSD cooling measures
    - The Kailash Engine — ExperimentTracker + Regression
    - Worked Example — CUPED on a Conversion Test
    - Try It Yourself
    - Cross-References
    - Reflection
- Lesson 2.8 — Capstone: Statistical Analysis Project
    - Why This Matters
    - The Full Pipeline
        - Step 1 — Load and explore
        - Step 2 — Feature engineering
        - Step 3 — Hypothesis
        - Step 4 — Tests
        - Step 5 — Model
        - Step 6 — Interpret
        - Step 7 — Present
    - Kailash Engines — Full Integration
    - Cross-References
    - Reflection

    - Module 2 Summary — Three Layers
    - Engines You Now Own
    - What's Next
- Appendix A — Formula Reference
    - Probability and Distributions (Lesson 2.1)
    - Estimation and Inference (Lesson 2.2)
    - Hypothesis Testing (Lesson 2.3)
    - A/B Testing (Lesson 2.4)
    - Linear Regression (Lesson 2.5)
    - Logistic Regression and ANOVA (Lesson 2.6)
    - CUPED and Causal Inference (Lesson 2.7)
- Appendix B — Glossary of Statistical Terms
- Appendix C — Common Mistakes Checklist
- Appendix D — Quick Decision Table
- Appendix E — Further Reading
- Appendix F — Self-Test: 20 Questions for Module 2

Module 2 — Statistical Mastery for Machine Learning and AI Success

> "The feature that killed a clinical trial looked perfect in every metric."

Module 1 taught you how to *see* data. Module 2 teaches you how to *reason* about it. Before a single machine-learning model is trained in Module 3, you need the vocabulary and the mathematical tools to answer questions like: *How confident are we in this number? Did the treatment actually work, or did we get lucky? Is this feature a genuine signal or a statistical artefact?* Without those tools, every downstream model is a guess wearing a confidence interval.

This chapter is a self-contained reference that you can read cover-to-cover or dip into when you need a refresher. Every formula is derived from first principles, every derivation is followed by a worked numerical example, and every concept is connected to the Kailash engines that implement it in practice. The running dataset is Singapore HDB resale prices, supplemented by small synthetic A/B tests and one causal inference case based on the 2021 ABSD cooling measures.

Learning Outcomes

By the end of this module you will be able to:

1. **Reason about uncertainty** using probability axioms, conditional probability, and Bayes' theorem. You will be able to translate a plain- English question ("if the test is positive, what's the chance I'm actually sick?") into a posterior update and explain the answer to a non-statistician.
2. **Estimate parameters** via maximum likelihood (MLE) and maximum a posteriori (MAP), derive the log-likelihood for the Normal and Binomial families from scratch, and explain when each method fails.
3. **Quantify uncertainty in estimates** using analytic standard errors, the bootstrap (percentile and BCa), and frequentist confidence intervals — and state their meaning *correctly* (a confidence interval is *not* "the 95% chance the parameter is in this range").
4. **Design and analyse A/B tests** with proper randomisation, power analysis, and SRM detection; interpret p-values without falling into the prosecutor's fallacy.
5. **Build linear and logistic regression models** from first principles. You will derive the OLS normal equations, the t-statistic on a coefficient, R-squared, the model F-test, and the logistic sigmoid / log-odds

link function without invoking a single black-box library.
6. **Apply ANOVA** as a natural generalisation of the two-sample t-test to three or more groups, including post-hoc corrections.
7. **Reduce A/B test variance** using CUPED (the single most impactful technique in modern online experimentation) and derive `Var(Y_adj) = Var(Y)(1 − ρ²)` from scratch.
8. **Make causal claims** from observational data using Difference-in- Differences and the parallel-trends assumption, and know when propensity methods are preferred.
9. **Integrate everything** into a capstone statistical analysis that loads data, engineers features, runs tests, builds a model, and presents findings to a non-technical audience.

Prerequisites

This chapter assumes you have completed Module 1. Specifically, you should be comfortable with:

- Loading data via the `MLFPDataLoader` and navigating parquet/CSV files.
- Polars DataFrames: `filter`, `group_by`, `agg`, `with_columns`, `select`.
- Running the Kailash `DataExplorer`, `PreprocessingPipeline`, and `ModelVisualizer` engines.
- Basic Python: functions, NumPy arrays, dictionaries, list comprehensions.
- High-school algebra and basic calculus (you should recognise a derivative when you see one; we will walk through every step).

If any of this feels shaky, pause here and return to the relevant M1 lesson. Module 2 introduces a lot of new formalism, and it is much easier to absorb new mathematics on top of a solid data-handling foundation than to learn both at once.

How to Read This Chapter

Every lesson follows a consistent structure:

1. **Why This Matters** — the business or real-world motivation. Start here if you want to know why the topic is useful before spending time on the maths.
2. **Core Concepts** — intuition in plain language followed by formal definitions. Worked examples show the concept in action on small numbers you can verify by hand.
3. **Mathematical Foundations** — full derivations from first principles. Every formula is derived, not stated. If a step is skipped, it's because the derivation appears in an earlier lesson and we cross-reference it.
4. **The Kailash Engine** — how the lesson's ideas are implemented in the kailash-ml platform. Every lesson is tied to at least one engine (`ExperimentTracker`, `FeatureEngineer`, `FeatureStore`, `ModelVisualizer`).
5. **Worked Example** — a full end-to-end problem using real Singapore data. You can run the code locally; imports and dataset names match `shared.MLFPDataLoader`.
6. **Try It Yourself** — three to five practice problems with solutions at the end of each section. The solutions are written out, not just answers, so you can compare your reasoning step by step.
7. **Cross-References** — forward links to later lessons and modules where the concept reappears in a more advanced form.
8. **Reflection** — a short prompt that asks you to connect the lesson to your own domain.

Three "layers" of depth are marked in-line:

- **FOUNDATIONS** (everyone reads this): plain-language intuition and a single worked example. If you only have 20 minutes, read just the foundations.
- **THEORY** (recommended): full derivations and multiple examples. This is the level you need to pass the module quiz and build sound models.
- **ADVANCED** (optional, marked clearly): edge cases, alternative formulations, and modern research extensions. Skip on first read; come back when the basics are automatic.

Common pitfalls are called out in boxed warnings like this:

> ⚠ **Pitfall:** The p-value is *not* the probability that the null hypothesis is true. We will come back to this repeatedly.

Everything assumes **Polars, not pandas**, and **Kailash engines, not raw sklearn**. That isn't style — it's structural. Kailash engines enforce guarantees (schema validation, point-in-time joins, experiment tracking) that make your analysis reproducible. Once you internalise the engines you will write less code, not more.

Lesson 2.1 — Probability and Bayesian Thinking

Why This Matters

Imagine you live in Singapore and you wake up one morning with a scratchy throat. You buy a COVID Antigen Rapid Test (ART) at the pharmacy. The box says the test is "99% accurate." You swab, wait 15 minutes, and see two red lines: *positive*. How worried should you be?

The natural reaction is "99% means I'm almost certainly infected." But that is wrong — sometimes spectacularly wrong. The correct answer depends on how common COVID is in the general population *right now*, a quantity that has nothing to do with the test. During a low-prevalence week in Singapore (say, 0.5% of the population actively infected), a positive ART test actually means you have roughly an **18% chance** of being infected (with typical ART sensitivity ~90%, specificity ~98%). During a surge week (say, 10% prevalence), the same positive result means you're about **83% likely** to be infected.

The difference is not the test. The difference is the *prior*. This lesson teaches you the single tool that separates good probabilistic reasoning from confident nonsense: **Bayes' theorem**.

You'll need this in ML for:

- **A/B test interpretation.** When a test shows that variant B has a 3% higher conversion rate, what's the probability that B is actually better than A? A frequentist p-value does not answer this question, but a Bayesian posterior does.
- **Fraud detection.** Given a transaction scored as "high risk," what is the probability it is actually fraud? Banks ask this every second.
- **Medical ML.** Every diagnostic model is a probability update over a prior that reflects disease prevalence.
- **Recommender systems.** When we see a user click three news articles about Formula 1, what is the posterior probability they are an F1 fan? How does this update change when they then click a cooking recipe?

At the end of this lesson you will be able to answer all four questions correctly, on real numbers, by hand.

Core Concepts — Foundations

What is probability?

A probability is a number between 0 and 1 that represents how likely we think an event is. There are two complementary interpretations, and a working data scientist uses both:

- **Frequentist interpretation.** If we repeat the experiment many times, probability is the long-run proportion of times the event happens. "The probability of heads is 0.5" means: if you flip a fair coin a

million times, roughly half a million will be heads.
- **Bayesian interpretation.** Probability is a *degree of belief* given the information you have. "There's a 70% chance it will rain tomorrow" doesn't require running tomorrow a thousand times; it's a statement about how confident you are given the current forecast, satellite data, and prior climate.

Both interpretations obey the same rules. The rules are known as the **Kolmogorov axioms**:

1. **Non-negativity.** `P(A) ≥ 0` for any event `A`. You cannot have negative probability.
2. **Normalisation.** `P(Ω) = 1`, where `Ω` is the entire sample space. Something in `Ω` is certain to happen.
3. **Additivity.** For mutually exclusive events `A` and `B` (they cannot both happen), `P(A ∪ B) = P(A) + P(B)`.

From these three axioms everything in this lesson — and in most of the module — can be derived.

A few immediate consequences you should remember:

- **Complement rule.** `P(not A) = 1 − P(A)`. If there is a 30% chance of rain, there is a 70% chance of no rain.
- **Union of two events.** `P(A ∪ B) = P(A) + P(B) − P(A ∩ B)`. You have to subtract the overlap once, because otherwise you count it twice.
- **The empty event.** `P(∅) = 0`. Nothing cannot happen.

Independent vs. dependent events

Two events `A` and `B` are **independent** if knowing that `B` happened tells you nothing about whether `A` happened. Formally:

```
P(A | B) = P(A)          (the notation P(A|B) means "probability of A given B")
```

Equivalently, for independent events:

```
P(A ∩ B) = P(A) × P(B)
```

Flipping a coin twice: whether the first flip lands heads tells you nothing about the second. `P(H on flip 2 | H on flip 1) = P(H on flip 2) = 0.5`.

Drawing two cards from a shuffled deck *without* replacement: the events are dependent. If the first card is the ace of spades, the probability that the second card is also the ace of spades drops to zero.

Practical test: ask yourself two questions.

1. **Does the order matter?** If you can always compute the answer by multiplying `P(A) × P(B)` regardless of which happens first, you are probably dealing with independence.
2. **Does one affect the sample space of the other?** If event `A` *removes* possibilities from the pool for `B` (draws a card, uses up inventory, influences someone's behaviour), the events are dependent.

Conditional probability

Conditional probability `P(A | B)` is defined as:

```
P(A | B) = P(A ∩ B) / P(B),     for P(B) > 0
```

In words: among all the outcomes where `B` happened, what fraction also have `A`? You restrict your view to the world in which `B` is true, and then ask what portion of that world also satisfies `A`.

Rearranging gives the **multiplication rule**:

```
P(A ∩ B) = P(B) × P(A | B) = P(A) × P(B | A)
```

We will use both forms constantly.

> ⚠ **Pitfall:** `P(A | B)` is *not* the same as `P(B | A)`. Mixing them up is called the **prosecutor's fallacy** and has, literally, put innocent people in jail. `P(match | innocent)` and `P(innocent | match)` can differ by orders of magnitude when the base rate is small.

Base rate fallacy (ART test example)

We already previewed this. Let's be precise. Define:

- `T+` = "the antigen test returns positive".
- `C+` = "the person actually has COVID".

The test manufacturer publishes two numbers:

- **Sensitivity** `P(T+ | C+) = 0.90`. Of people who actually have COVID, 90% test positive.
- **Specificity** `P(T− | C−) = 0.98`. Of people who don't have COVID, 98% correctly test negative. (Equivalently, `P(T+ | C−) = 0.02`, the false-positive rate.)

What we *want* is `P(C+ | T+)`, the probability that, given a positive test, the person actually has COVID. Intuitively, "the test is 98% specific, so my chance of being sick is 98%." This is wrong. We need Bayes' theorem, and we need to know the **prior** `P(C+)` — the base rate of COVID in the population you sampled from.

Suppose the current prevalence is 0.5% (`P(C+) = 0.005`). Then:

```
P(T+) = P(T+ | C+) × P(C+) + P(T+ | C−) × P(C−)
      = 0.90 × 0.005 + 0.02 × 0.995
      = 0.0045 + 0.01990
      = 0.02440

P(C+ | T+) = P(T+ | C+) × P(C+) / P(T+)
           = 0.90 × 0.005 / 0.02440
           = 0.00450 / 0.02440
           ≈ 0.1844
```

A positive test means there's roughly an **18% chance** you actually have COVID. The other 82% of the time you're a false positive — because the base rate is so low that *most* positive tests come from the 99.5% of people who don't have COVID but occasionally trip the 2% false-positive rate. One in five positives is a true positive; four in five are false alarms.

Now run the same calculation with 10% prevalence:

```
P(T+) = 0.90 × 0.10 + 0.02 × 0.90
      = 0.090 + 0.018
      = 0.108

P(C+ | T+) = 0.90 × 0.10 / 0.108 ≈ 0.833
```

A positive test now means an 83% chance of being sick — a very different story. Same test, same sensitivity, same specificity; only the prior has changed.

> ⚠ **Pitfall — Base rate fallacy.** Never interpret a conditional probability without thinking about the base rate. In ML, the base rate is usually the class prior `P(y = 1)` — the rate of fraud, churn, disease, clicks, whatever you're predicting.

Mathematical Foundations — Theory

Deriving Bayes' theorem

Bayes' theorem is a two-line consequence of the multiplication rule. Start with the two ways to write `P(A ∩ B)`:

```
P(A ∩ B) = P(A) × P(B | A)          (condition on A first)
P(A ∩ B) = P(B) × P(A | B)          (condition on B first)
```

Since both equal `P(A ∩ B)`, we can set the right-hand sides equal:

```
P(A) × P(B | A) = P(B) × P(A | B)
```

Divide both sides by `P(B)`:

```
P(A | B) = P(B | A) × P(A) / P(B)
```

That's Bayes' theorem. The components have names that we will use forever:

- `P(A)` — the **prior**. Your belief about `A` *before* seeing the data.
- `P(B | A)` — the **likelihood**. How probable the data is, given the hypothesis.
- `P(B)` — the **evidence** (or marginal likelihood). The overall probability of observing `B`, marginalised over every possible state of the world.
- `P(A | B)` — the **posterior**. Your updated belief about `A` *after* observing `B`.

A pattern you should memorise:

```
posterior = (likelihood × prior) / evidence
```

The denominator is usually computed via the **law of total probability**:

```
P(B) = P(B | A) × P(A) + P(B | ¬A) × P(¬A)
```

You can extend this to any partition of the sample space. If $A_1, A_2, \ldots, A_k$ are mutually exclusive and cover everything:

$$P(B) = \Sigma_i \; P(B \mid A_i) \times P(A_i)$$

Expected value

Given a random variable `X` that takes values $x_1, x_2, \ldots, x_n$ with probabilities $p_1, p_2, \ldots, p_n$, the **expected value** is:

$$E[X] = \Sigma_i \; p_i \times x_i$$

Think of `E[X]` as the probability-weighted average of all possible outcomes. If you roll a fair six-sided die:

```
E[X] = (1/6)(1) + (1/6)(2) + (1/6)(3) + (1/6)(4) + (1/6)(5) + (1/6)(6)
     = (1 + 2 + 3 + 4 + 5 + 6) / 6
     = 21 / 6
     = 3.5
```

You will never roll a 3.5, but over many rolls your average will approach 3.5. That's the **Law of Large Numbers** (LLN), which we'll meet formally in Lesson 2.2.

Expectation is linear:

```
E[aX + bY] = a × E[X] + b × E[Y]
```

This holds even if `X` and `Y` are dependent. Linearity of expectation is one of the most useful identities in probability — it lets you break down complicated sums without worrying about independence.

For a continuous random variable with density `f(x)`, the sum becomes an integral:

```
E[X] = ∫ x × f(x) dx
```

We will not compute many integrals by hand in this module, but recognising the pattern helps you read papers and library docs.

Variance — two equivalent forms

Variance measures the spread of a distribution around its mean. For a discrete random variable:

```
Var(X) = E[(X − μ)²]        where μ = E[X]
```

This is the average squared distance from the mean. Why squared? Because if we used unsigned distance `|X − μ|`, the derivatives would misbehave at the minimum (the absolute value isn't differentiable at zero), and almost every theorem in statistics would become harder to prove. Squared distance is smooth, additive for independent variables, and leads to the Normal distribution as a natural limit via the CLT.

There's a second form that is often easier to compute:

```
Var(X) = E[X²] − (E[X])² = E[X²] − μ²
```

Let's derive it. Starting from the definition:

```
Var(X) = E[(X − μ)²]
       = E[X² − 2Xμ + μ²]         (expand the square)
       = E[X²] − 2μ × E[X] + μ²   (linearity of expectation; μ is a constant)
       = E[X²] − 2μ² + μ²         (since E[X] = μ)
       = E[X²] − μ²
```

Done. Both forms are correct, but the second is numerically stable only when `E[X²]` and `μ²` are not dangerously close in magnitude. In practice, Polars and NumPy use a two-pass algorithm that's both exact and stable.

Standard deviation is `σ = √Var(X)`. It has the same units as `X`, which makes it easier to interpret than variance.

Numerical example — HDB prices

Suppose we have five small HDB sales (in thousands of SGD): 380, 420, 450, 500, 600. Compute the mean and variance:

```
μ = (380 + 420 + 450 + 500 + 600) / 5 = 2350 / 5 = 470

(380 − 470)² = 8100
(420 − 470)² = 2500
(450 − 470)² =  400
(500 − 470)² =  900
(600 − 470)² = 16900
sum of squared deviations = 28800

Var (population form) = 28800 / 5 = 5760
σ = √5760 ≈ 75.9
```

If we treat the five values as a *sample* rather than a population, we divide by `n − 1 = 4` instead of `n = 5`:

```
Var (sample form) = 28800 / 4 = 7200
s = √7200 ≈ 84.9
```

We'll explain why `n - 1` in Lesson 2.2 (it's called **Bessel's correction**). For now, remember: *population* variance divides by `n`, *sample* variance divides by `n - 1`.

Distributions — the statistics theme park

Each common distribution models a different kind of uncertainty. You will meet five in this module.

Normal (Gaussian). `X ~ N(μ, σ²)`. The bell-shaped curve. Heights, weights, measurement errors, sums of many small effects, HDB prices (after log transformation), stock returns (to first approximation). Its density is:

```
f(x) = (1 / √(2π σ²)) × exp(−(x − μ)² / (2 σ²))
```

Two parameters: mean μ and variance σ². The CLT (Lesson 2.2) tells us that *sums of independent random variables* tend toward Normal regardless of their individual distributions, which is why it is everywhere.

Beta. `X ~ Beta(α, β)`. A distribution on `[0, 1]`. Perfect for modelling probabilities, conversion rates, proportions. The density is:

```
f(p) ∝ p^(α−1) × (1 − p)^(β−1)
```

The parameters α and β behave like "successes + 1" and "failures + 1" counts. Beta is the **conjugate prior** to the Binomial distribution, so updating a Beta prior with Binomial data just means adding counts.

Binomial. `X ~ Binomial(n, p)`. The number of successes in `n` independent Bernoulli trials, each with success probability `p`. Example: number of users who clicked in an A/B test cell of 1000 impressions.

Poisson. `X ~ Poisson(λ)`. Counts of rare events per unit time or space: arrivals at a queue, typos per page, traffic accidents per month. The parameter λ is both the mean and the variance.

Exponential. `X ~ Exp(λ)`. The waiting time between Poisson events. Memoryless: the distribution of additional waiting time given that you've already waited is identical to the original. `E[X] = 1/λ`.

Uniform. `X ~ Uniform(a, b)`. Every value in `[a, b]` is equally likely. Used as a default non-informative prior and as the output of most random number generators (before transformation).

Sampling bias — the friendship paradox

Sampling bias is what happens when the way you select observations is correlated with what you're trying to measure. The most famous example is the **friendship paradox**: on average, your friends have more friends than you.

This is *not* a self-esteem issue. It's a statistical fact. When you sample a random person and ask "how many friends do you have?", you get the average over people. When you sample a random *friend* (by picking a random person, then picking one of their friends uniformly), you over-sample popular people — because popular people are *more often* someone's friend. The average of that biased sample is larger.

This matters in ML because we constantly deal with biased samples:

- **Click-stream data** over-samples active users.
- **Survey data** over-samples people willing to respond.
- **Medical trial data** over-samples people sick enough to seek care.

The correction is to either debias the sample explicitly (using inverse propensity weights, which we'll meet in Lesson 2.7) or to use a different sampling scheme that avoids the bias in the first place.

The Kailash Engine — ExperimentTracker

`ExperimentTracker` is kailash-ml's experiment metadata store. It doesn't run the statistics itself — that's your code, or other engines — but it records every parameter, metric, and artifact so experiments are **reproducible** and **comparable** months later.

```
from kailash_ml import ExperimentTracker

tracker = ExperimentTracker()

with tracker.start_run(name="covid_ART_bayesian_update") as run:
    run.log_param("prior_prevalence", 0.005)
    run.log_param("sensitivity", 0.90)
    run.log_param("specificity", 0.98)

    posterior = 0.90 * 0.005 / (0.90 * 0.005 + 0.02 * 0.995)
    run.log_metric("posterior_prob_covid_given_positive", posterior)
```

Why is this worth doing for such a simple calculation? Because in a week from now, the prevalence will be different, the test version will be different, and you won't remember which numbers went into which analysis. `ExperimentTracker` makes a durable trail of *exactly* which prior produced *exactly* which posterior. We'll use it in every lesson from here on.

Worked Example — Full Bayesian Update on HDB Prices

Suppose you are a housing analyst at a Singapore bank. Before looking at any data, your manager says "4-room HDB flats in 2024 cost about SGD 500,000, give or take SGD 25,000." This is a prior belief:

$$\mu \sim N(\mu_0, \sigma_0^2), \quad \mu_0 = 500_000, \quad \sigma_0 = 25_000$$

You then pull $n = 1000$ recent 4-room transactions from data.gov.sg. The sample mean is $\bar{x} = 540_000$ with sample standard deviation $s = 80_000$. Treating the individual observations as Normal with known variance $\sigma^2 = s^2 = 6.4e9$, we can use the **Normal-Normal conjugate update**.

Derivation. If $\mu \sim N(\mu_0, \sigma_0^2)$ and each observation $x_i \sim N(\mu, \sigma^2)$ (independent given μ), then the posterior is:

$$\mu \mid x_1, \ldots, x_n \sim N(\mu_n, \sigma_n^2)$$

$$\text{where:}$$
$$1/\sigma_n^2 = 1/\sigma_0^2 + n/\sigma^2$$
$$\mu_n = \sigma_n^2 \times (\mu_0/\sigma_0^2 + (n \times \bar{x})/\sigma^2)$$

Plug in numbers:

$$1/\sigma_0^2 = 1 / (25_000^2) = 1 / 625_000_000 = 1.6e{-9}$$
$$n/\sigma^2 = 1000 / 6.4e9 = 1.5625e{-7}$$
$$1/\sigma_n^2 = 1.6e{-9} + 1.5625e{-7} \approx 1.5785e{-7}$$
$$\sigma_n^2 \approx 6.335e6$$
$$\sigma_n \approx 2517$$

$$\mu_0/\sigma_0^2 = 500_000 \times 1.6e{-9} = 0.0008$$
$$n \times \bar{x}/\sigma^2 = 1000 \times 540_000 / 6.4e9 = 0.084375$$
$$\text{sum} = 0.085175$$

$\mu_n = \sigma_n^2 \times 0.085175 = 6.335e6 \times 0.085175 \approx 539_573$

So after seeing the data, your posterior mean is about **SGD 539,573** with standard deviation about **SGD 2,517**. Three observations:

1. The posterior is pulled almost all the way to the data, because with `n = 1000` the data dominates the prior. This is exactly the **Bernstein-von Mises phenomenon**: with enough data, any reasonable prior converges to the same posterior.
2. The posterior standard deviation (2,517) is much smaller than either the prior (25,000) or the sample standard deviation of individual prices (80,000). Averaging lots of samples sharpens our estimate of the *mean*, even though individual prices stay noisy.
3. The 95% **credible interval** (the Bayesian analog of a confidence interval) is $\mu_n \pm 1.96 \times \sigma_n \approx$ `[534,640, 544,506]`. You can literally say "I'm 95% sure the true mean is in this range" — which, as we'll see in Lesson 2.2, you *cannot* say about a frequentist CI.

Code:

```
import numpy as np
from kailash_ml import ExperimentTracker, ModelVisualizer
import polars as pl

from shared import MLFPDataLoader

loader = MLFPDataLoader()
hdb = loader.load("mlfp01", "hdb_resale.parquet")

prices = (
    hdb.filter(pl.col("flat_type") == "4 ROOM")
       .filter(pl.col("month").str.to_date("%Y-%m") >= pl.date(2024, 1, 1))
       ["resale_price"].to_numpy().astype(float)
)
n = len(prices)
x_bar = prices.mean()
s = prices.std(ddof=1)

mu0, sigma0 = 500_000.0, 25_000.0
sigma = s  # assume known variance equals sample variance
post_var = 1.0 / (1.0/sigma0**2 + n/sigma**2)
post_mean = post_var * (mu0/sigma0**2 + (n * x_bar)/sigma**2)
post_sd = np.sqrt(post_var)

print(f"posterior mean  = {post_mean:,.0f}")
print(f"posterior sd    = {post_sd:,.0f}")
print(f"95% credible IV = [{post_mean - 1.96*post_sd:,.0f}, "
      f"{post_mean + 1.96*post_sd:,.0f}]")

with ExperimentTracker().start_run(name="hdb_bayes_update") as run:
    run.log_param("prior_mu0", mu0)
    run.log_param("prior_sigma0", sigma0)
    run.log_param("sample_size", n)
    run.log_metric("posterior_mean", post_mean)
    run.log_metric("posterior_sd", post_sd)
```

The `ModelVisualizer` overlay of prior, likelihood, and posterior (which the exercise walks through) shows three curves: a wide bell centred at 500K (prior), a narrow bell centred at 540K (likelihood of the sample mean), and an even narrower bell that almost perfectly overlaps the likelihood (posterior). The visual makes clear that the data has overwhelmed the prior.

Try It Yourself

Problem 1 — Email spam. A spam filter marks 95% of actual spam as spam (sensitivity) and mistakenly marks 1% of legitimate email as spam (false positive rate). You know that 20% of your incoming mail is spam. An email is marked as spam. What is the probability it's actually spam?

Solution. Let `S` = "is spam", `M` = "marked spam".

```
P(S | M) = P(M | S) × P(S) / P(M)
P(M) = 0.95 × 0.20 + 0.01 × 0.80 = 0.190 + 0.008 = 0.198
P(S | M) = (0.95 × 0.20) / 0.198 = 0.190 / 0.198 ≈ 0.9596
```

Roughly 96%. This is a much higher "posterior precision" than the COVID test because spam is common enough (20%) that the prior doesn't fight the likelihood as hard.

Problem 2 — Two children. A colleague says, "I have two children. At least one is a boy." What is the probability both are boys?

Solution. Label the children by age. The four equally likely combinations are BB, BG, GB, GG. Conditioning on "at least one boy" eliminates GG, leaving BB, BG, GB. Only BB has two boys, so:

```
P(BB | at least one B) = 1 / 3
```

Not `1/2`. This surprises almost everyone the first time. The subtle point is that "at least one boy" is a constraint on the *joint* distribution, not on a specific child.

Problem 3 — Rolling dice. You roll two fair six-sided dice. Let `A` = "sum is 7" and `B` = "first die is 3". Are `A` and `B` independent?

Solution. `P(A) = 6/36 = 1/6` (the six combinations that sum to 7). `P(B) = 6/36 = 1/6`. `P(A ∩ B) = 1/36` (only 3+4 works). Check:

```
P(A) × P(B) = (1/6)(1/6) = 1/36 = P(A ∩ B)   ✓
```

They are independent. Intriguingly, if we changed `A` to "sum is 8", the answer changes: `P(A) = 5/36`, `P(B) = 6/36`, `P(A ∩ B) = 1/36`. Then `P(A) × P(B) = 30/1296 ≠ 36/1296`. Not independent.

Problem 4 — HDB prior. You believe 3-room flats cost `N(400K, 30K²)` a priori. You observe 500 transactions with sample mean 420K and sample SD 60K. Compute the posterior mean and SD for the true mean price.

Solution. Plug into the Normal-Normal formula:

```
1/σₙ² = 1/30_000² + 500/60_000² = 1.111e−9 + 1.389e−7 ≈ 1.400e−7
σₙ² ≈ 7.143e6, σₙ ≈ 2672

μ₀/σ₀² = 400_000 × 1.111e−9 = 0.000444
(n × x̄)/σ² = 500 × 420_000 / 3.6e9 = 0.05833
μₙ = 7.143e6 × (0.000444 + 0.05833) ≈ 419_844
```

Posterior mean ≈ SGD 419,844, posterior SD ≈ SGD 2,672.

Problem 5 — Whose base rate? A medical device scores 92% sensitivity and 98% specificity for a rare disease present in 1 in 10,000 people. A patient tests positive. What is the probability they have the disease?

Solution.

```
P(D) = 0.0001
P(T+) = 0.92 × 0.0001 + 0.02 × 0.9999 = 0.000092 + 0.019998 = 0.02009
P(D | T+) = 0.000092 / 0.02009 ≈ 0.00458
```

Roughly 0.46%. Even with a very accurate test, the rare base rate dominates: well over 99% of positives are false alarms. In practice, medical systems handle this by requiring confirmatory tests — Bayes' theorem again, this time chained.

Cross-References

- **Lesson 2.2** extends conjugate updating into maximum likelihood and MAP estimation.
- **Lesson 2.3** uses the Binomial likelihood for permutation-based A/B testing.
- **Lesson 2.5** will show how a regression coefficient's t-statistic is a frequentist analog of the conjugate update we just derived.
- **Module 3** uses priors as regularisers: L2 regularisation is a Gaussian prior on coefficients, L1 is a Laplace prior.
- **Module 6** uses Bayesian updating for alignment preference learning (DPO compares likelihoods of preferred vs rejected responses).

Reflection

Pick one decision from your own domain that involves updating beliefs when new information arrives. Write down:

1. The prior probability of the event in question.
2. The sensitivity and false positive rate of whatever signal you use.
3. The posterior probability after a positive signal.

If your domain never quantifies these things, ask *why not*. That's Module 2 in one sentence.

Lesson 2.2 — Parameter Estimation and Inference

Why This Matters

Every model you will ever train is built on this question: *given some data, what parameter values best describe the process that generated it?* A linear regression wants the slope and intercept. A logistic regression wants a vector of coefficients. A Normal distribution wants a mean and variance. A deep neural network wants weights for every edge.

The answer is almost always: **maximum likelihood estimation (MLE)**, possibly with a prior turning it into MAP. MLE is the single most important idea in this module. Once you understand it, the rest of classical statistics — t-tests, regression, even logistic regression — falls out as special cases.

This lesson covers:

1. Population parameters vs sample statistics (and why confusing them is the most common statistical mistake).
2. The Law of Large Numbers and the Central Limit Theorem — the two pillars of frequentist inference.
3. Confidence intervals: what they actually mean (hint: *not* what you think).
4. Maximum likelihood estimation from first principles: write the log-likelihood, take the derivative, set it to zero.

5. Full MLE derivations for the Normal and Binomial distributions.
6. MAP estimation as a Bayesian generalisation.
7. When MLE fails and what to do instead.

Core Concepts — Foundations

Population vs sample

A **population** is the complete set of entities you care about. The population mean μ and population variance σ^2 are fixed (though usually unknown) numbers. The population of "all 4-room HDB flats sold in Singapore in 2024" has a specific average price; you could in principle compute it exactly by including every sale.

A **sample** is a subset you actually observe. The sample mean $\bar{x}$ and sample variance s^2 are computed from the sample. They are *random* — a different sample would give different values.

The entire point of statistics is to reason about the population (what we care about) using a sample (what we have). Everything else is bookkeeping.

> ⚠ **Pitfall — Greek letters vs Latin letters.** Population parameters get Greek letters: μ, σ, β, ρ. Sample statistics get Latin letters with hats or bars: $\bar{x}$, s, $\hat{\beta}$, r. Confusing them is the mark of someone who has not internalised the sampling model.

Sampling distribution

Imagine you could repeat your experiment (draw a sample, compute a statistic) infinitely many times. The distribution of the resulting statistics is the **sampling distribution**. It is the engine behind every confidence interval and every p-value.

Example. Suppose the true mean 4-room HDB price is $\mu = 540_000$ with population standard deviation $\sigma = 80_000$. If you repeatedly draw samples of size $n = 100$ and compute $\bar{x}$, how are those $\bar{x}$ values distributed?

The Central Limit Theorem (below) tells us: approximately $N(540_000, (80_000 / \sqrt{100})^2) = N(540_000, 8000^2)$. So:

- 68% of your sample means will land within `[532K, 548K]`.
- 95% will land within `[524K, 556K]`.
- The *individual* prices have SD 80K, but the *sample mean* has SD only 8K. The mean is far more precise than any one observation.

Law of Large Numbers (LLN)

As $n \to \infty$, the sample mean $\bar{x}$ converges to the population mean μ. Formally, for any $\varepsilon > 0$:

$$P(|\bar{x} - \mu| > \varepsilon) \to 0 \quad \text{as} \quad n \to \infty$$

The LLN is why your 1,000-flat sample gives a better estimate than your 10-flat sample. It is the mathematical formalisation of "more data is more accurate."

Central Limit Theorem (CLT)

The LLN tells you *where* the sample mean goes. The CLT tells you *how fast* and *in what shape*. For any population with finite variance σ^2, the distribution of $\bar{x}$ is approximately Normal for large n:

```
x̄  ≈  N(μ, σ² / n)
```

Equivalently, the standardised version is approximately standard Normal:

```
Z = (x̄ − μ) / (σ / √n)  ≈  N(0, 1)
```

Three things to notice:

1. **The shape is Normal regardless of the population shape.** This is the "miracle" of the CLT. HDB prices are right-skewed with a long tail, but the sampling distribution of the mean across many samples is still nearly Normal.
2. **The standard deviation of the sample mean is `σ/√n`**, called the **standard error** (SE). It shrinks with `√n`, which is why you need four times as much data to halve your uncertainty.
3. **Finite variance is required.** If the population has heavy tails with infinite variance (Cauchy, some power-law distributions), the CLT does not hold and your intuition about sample means is wrong.

Confidence intervals — what they really mean

A **95% confidence interval** for μ, under the CLT, is:

```
CI = [x̄ − 1.96 × (σ / √n),  x̄ + 1.96 × (σ / √n)]
```

(If σ is unknown, replace it with `s` and use the t-distribution critical value instead of 1.96; more on that in Lesson 2.3.)

Now the correct interpretation. If you repeated the experiment many times and built a 95% CI each time, about 95% of those intervals would contain μ. The CI is a property of the **procedure**, not of any single interval you build.

> ⚠ **Pitfall — The wrong interpretation.** A 95% confidence interval does **not** mean "there is a 95% probability that μ is in this specific interval." μ is a fixed unknown; either it is in your interval or it isn't. The randomness lives in the sample, not in μ. To say "there is a 95% chance μ is in this range" you need a Bayesian **credible** interval.

Numerical example. Suppose `n = 100`, `x̄ = 540_000`, `s = 80_000`. The standard error is `80_000 / √100 = 8000`. The 95% CI is:

```
[540_000 − 1.96 × 8000,  540_000 + 1.96 × 8000]
= [524_320, 555_680]
```

If you drew 100 samples of `n = 100` and built a 95% CI for each, you would expect about 95 of them to contain the true μ. This one interval either does or doesn't; we don't know which.

Bessel's correction — why `n − 1`

The population variance is:

```
σ² = (1/N) Σᵢ (xᵢ − μ)²
```

If we knew μ, we could estimate σ² by plugging in the sample:

```
(1/n) Σᵢ (xᵢ − μ)²
```

This is unbiased. But we don't know μ; we use x̄ instead. Here's the problem: x̄ is the sample value that *minimises* the sum of squared deviations. Using it in place of μ systematically under-estimates the true variance — the sample hugs its own mean too tightly.

The fix is to divide by $n - 1$ instead of n:

$$s^2 = (1 / (n - 1)) \Sigma_i (x_i - \bar{x})^2$$

Why exactly $n - 1$? Intuitively: once you know $n - 1$ of the deviations $(x_i - \bar{x})$, the last one is forced by the constraint $\Sigma_i (x_i - \bar{x}) = 0$. You've used one "degree of freedom" to estimate $\bar{x}$, leaving $n - 1$ for variance.

Formal derivation (THEORY). We want $E[s^2] = \sigma^2$. Write:

$$\begin{aligned}\Sigma_i (x_i - \bar{x})^2 &= \Sigma_i ((x_i - \mu) - (\bar{x} - \mu))^2 \\ &= \Sigma_i (x_i - \mu)^2 - 2(\bar{x} - \mu) \Sigma_i (x_i - \mu) + n(\bar{x} - \mu)^2\end{aligned}$$

Now $\Sigma_i (x_i - \mu) = n(\bar{x} - \mu)$, so:

$$\begin{aligned}&= \Sigma_i (x_i - \mu)^2 - 2n(\bar{x} - \mu)^2 + n(\bar{x} - \mu)^2 \\ &= \Sigma_i (x_i - \mu)^2 - n(\bar{x} - \mu)^2\end{aligned}$$

Take expectations:

$$\begin{aligned}E[\Sigma_i (x_i - \bar{x})^2] &= \Sigma_i E[(x_i - \mu)^2] - n \times E[(\bar{x} - \mu)^2] \\ &= n\sigma^2 - n \times (\sigma^2/n) \quad \text{(the last term is } Var(\bar{x}) = \sigma^2/n \text{ by CLT logic)} \\ &= n\sigma^2 - \sigma^2 = (n - 1) \sigma^2\end{aligned}$$

Dividing by $(n - 1)$ gives $E[s^2] = \sigma^2$. Dividing by n would give $((n-1)/n) \sigma^2 < \sigma^2$, systematically too small. That's Bessel's correction: dividing by $n - 1$ restores unbiasedness.

Mathematical Foundations — MLE from First Principles

The idea in one sentence

> **MLE picks the parameter value that makes the observed data most likely.**

Formally: given data $x_1, \ldots, x_n$ assumed to come i.i.d. from a distribution with density $f(x; \theta)$, the **likelihood** is:

$$L(\theta) = f(x_1; \theta) \times f(x_2; \theta) \times \ldots \times f(x_n; \theta) = \prod_i f(x_i; \theta)$$

The **log-likelihood** is:

$$\ell(\theta) = \log L(\theta) = \Sigma_i \log f(x_i; \theta)$$

We take logs for two reasons:

1. Products of small probabilities *underflow* in floating-point arithmetic. Sums don't.
2. The derivative of a sum is simpler than the derivative of a product. The logarithm turns MLE into an additive problem.

The **MLE** is:

$$\hat{\theta}_MLE = \text{argmax}_\theta \ \ell(\theta)$$

You find it by taking derivatives and setting them to zero (and checking the second derivative is negative to confirm a maximum).

Derivation 1 — MLE for the Normal distribution

Suppose $x_1, \ldots, x_n$ are i.i.d. $N(\mu, \sigma^2)$. The Normal density is:

$$f(x; \mu, \sigma^2) = (1 / \sqrt{(2\pi \sigma^2)}) \times \exp(-(x - \mu)^2 / (2\sigma^2))$$

Taking the log:

$$\log f(x; \mu, \sigma^2) = -(1/2) \log(2\pi) - (1/2) \log(\sigma^2) - (x - \mu)^2 / (2\sigma^2)$$

Sum over i:

$$\ell(\mu, \sigma^2) = -(n/2) \log(2\pi) - (n/2) \log(\sigma^2) - (1 / (2\sigma^2)) \times \Sigma_i (x_i - \mu)^2$$

Solve for μ. Take the partial derivative with respect to μ:

$$\partial\ell/\partial\mu = (1/\sigma^2) \times \Sigma_i (x_i - \mu)$$

Set to zero:

$$\Sigma_i (x_i - \hat{\mu}) = 0 \quad \Rightarrow \quad n \times \hat{\mu} = \Sigma_i x_i \quad \Rightarrow \quad \hat{\mu} = \bar{x}$$

So the MLE of the mean is the **sample mean**. Totally unsurprising, but now we know *why*: it is the value that maximises the joint probability of the observed data under the Normal model.

Solve for σ^2. Take the partial derivative with respect to σ^2:

$$\partial\ell/\partial\sigma^2 = -n / (2\sigma^2) + (1 / (2\sigma^4)) \times \Sigma_i (x_i - \hat{\mu})^2$$

Set to zero and solve:

$$n / (2\hat{\sigma}^2) = (1 / (2\hat{\sigma}^4)) \times \Sigma_i (x_i - \hat{\mu})^2$$
$$n \times \hat{\sigma}^2 = \Sigma_i (x_i - \hat{\mu})^2$$
$$\hat{\sigma}^2_MLE = (1/n) \times \Sigma_i (x_i - \bar{x})^2$$

The MLE of the variance divides by n, *not* $n - 1$. That makes it slightly biased (see Bessel's correction). MLE estimators can be biased; we accept that trade-off in exchange for other desirable properties (asymptotic efficiency and consistency).

Derivation 2 — MLE for the Binomial / Bernoulli

Suppose you run n Bernoulli trials and observe k successes. The likelihood is:

$$L(p) = (n \text{ choose } k) \times p^k \times (1 - p)^{(n - k)}$$

Dropping the constant n choose k (it doesn't depend on p), the log-likelihood is:

$$\ell(p) = k \times \log(p) + (n - k) \times \log(1 - p)$$

Differentiate:

$$\partial\ell/\partial p = k/p - (n - k)/(1 - p)$$

Set to zero:

$$k(1 - \hat{p}) = (n - k) \times \hat{p}$$
$$k - k \times \hat{p} = n \times \hat{p} - k \times \hat{p}$$
$$k = n \times \hat{p}$$
$$\hat{p}_MLE = k / n$$

Again unsurprising: the MLE of a proportion is the sample proportion. Again, now we know why.

Fisher information (THEORY)

The **Fisher information** `I(θ)` at a parameter value is a measure of how curved the log-likelihood is at the MLE. Sharp curvature = lots of information (the log-likelihood pinches the estimate tightly); flat curvature = little information (almost any `θ` is equally plausible).

For a single observation:

```
I(θ) = −E[∂²ℓ/∂θ²]
```

For the Normal with known `σ²`, the per-observation Fisher information for `μ` is:

```
I(μ) = 1 / σ²
```

and for a sample of size `n`:

```
I_n(μ) = n / σ²
```

The **Cramér-Rao bound** says no unbiased estimator can have variance smaller than `1 / I_n(θ)`. For the Normal mean, this bound is `σ²/n` — and the sample mean achieves it exactly. The sample mean is **asymptotically efficient**; you cannot do better with an unbiased estimator.

MAP — adding a prior (THEORY)

Maximum a posteriori estimation is MLE with a prior. Starting from Bayes:

```
P(θ | data) ∝ P(data | θ) × P(θ)
```

Taking logs:

```
log P(θ | data) = ℓ(θ) + log P(θ) + const
```

The **MAP estimator** is:

```
θ̂_MAP = argmax_θ (ℓ(θ) + log P(θ))
```

Compare to MLE (`argmax_θ ℓ(θ)`) — the only difference is the `log P(θ)` term, which acts as a **regulariser** that pulls the estimate toward the prior.

Connection to L2 regularisation. If you place a Gaussian prior `β ~ N(0, τ²)` on a regression coefficient, then:

```
log P(β) = −β² / (2τ²) + const
```

Maximising the sum of log-likelihood plus this prior is equivalent to minimising:

```
− ℓ(β) + β² / (2τ²)
```

which is exactly ordinary least squares with an `β²` penalty — i.e. **ridge regression**. We will return to this in Module 3.

Connection to L1 regularisation. A Laplace prior `β ~ Laplace(0, b)` gives `log P(β) = −|β|/b + const`, which leads to **lasso regression**.

When MLE fails (ADVANCED)

MLE has beautiful asymptotic properties but can misbehave in finite samples:

1. **Small `n`, high dimension.** Classical regime: `n → ∞` with `p` fixed. If `p` is large relative to `n`, MLE variances explode. Solution: regularise (MAP) or reduce dimensionality.
2. **Multimodal likelihood.** Sometimes the log-likelihood has multiple peaks of roughly equal height. Any one peak is an MLE but none is uniquely good. Solution: Bayesian model averaging or explicit mode-finding.
3. **Model misspecification.** If the data don't come from the model family you're fitting, MLE converges to the *Kullback-Leibler closest* member of the family, which may be a bad approximation. Solution: expand the model family or use robust methods.
4. **Infinite likelihood.** A Gaussian mixture with a component whose variance is allowed to shrink to zero around a single data point has infinite likelihood there. Solution: constrain the parameter space or use a prior.
5. **Non-identifiable models.** Two different parameter values produce the same distribution. MLE is not unique. Solution: reparametrise or impose identifiability constraints.

The Kailash Engine — ExperimentTracker + ModelVisualizer

For MLE work we use two engines in combination. `ExperimentTracker` logs the parameters and metrics; `ModelVisualizer` renders the log-likelihood curve so you can see whether the maximum is sharp (lots of information) or flat (not much).

```
from kailash_ml import ExperimentTracker, ModelVisualizer
from scipy import stats
import numpy as np

x = prices  # from the HDB dataset

mu_grid = np.linspace(x.mean() - 3 * x.std(ddof=1) / np.sqrt(len(x)),
                      x.mean() + 3 * x.std(ddof=1) / np.sqrt(len(x)),
                      200)
loglik = [stats.norm.logpdf(x, loc=mu, scale=x.std(ddof=0)).sum()
          for mu in mu_grid]

viz = ModelVisualizer()
viz.line(mu_grid, loglik, xlabel="mu", ylabel="log-likelihood",
         title="Log-likelihood for HDB mean price")

with ExperimentTracker().start_run(name="hdb_mle_normal") as run:
    run.log_param("n", len(x))
    run.log_metric("mu_mle", float(x.mean()))
    run.log_metric("sigma_mle", float(x.std(ddof=0)))
    run.log_metric("fisher_info", float(len(x) / x.var()))
```

Worked Example — MLE, Fisher Information, and a Confidence Interval

Use the same 4-room HDB data (2024) as Lesson 2.1. Suppose `n = 1000`, `x̄ = 540_000`, `s = 80_000`.

MLE of μ: `540_000` (the sample mean).

MLE of σ²: roughly `s² × (n − 1) / n ≈ 80_000² × 0.999 ≈ 6.395e9`.

Fisher information (per-parameter μ): `n / σ² ≈ 1000 / 6.4e9 ≈ 1.5625e−7`.

Cramér-Rao bound on `Var(μ̂)`: `1 / I_n ≈ 6.4e6`, so `SE(μ̂) ≈ √6.4e6 ≈ 2530`.

95% CI for μ: `540_000 ± 1.96 × 2530 ≈ [535_042, 544_958]`.

Compare to Lesson 2.1's posterior credible interval, which was `[534_640, 544_506]`. The frequentist CI and the Bayesian credible interval are almost identical because the data dominates the prior. Bernstein-von Mises in action.

Try It Yourself

Problem 1 — Exponential MLE. Derive the MLE for λ in the exponential distribution `f(x; λ) = λ × e^(−λx)` from `n` i.i.d. observations.

Solution. Log-likelihood:

```
ℓ(λ) = Σᵢ log(λ) + Σᵢ (−λxᵢ) = n log λ − λ Σᵢ xᵢ
```

Derivative and zero:

```
∂ℓ/∂λ = n/λ − Σᵢ xᵢ = 0
λ̂ = n / Σᵢ xᵢ = 1 / x̄
```

So the MLE of the rate is the reciprocal of the sample mean (since `E[X] = 1/λ`, this just inverts the moment equation).

Problem 2 — Poisson MLE. Derive the MLE for λ in the Poisson distribution `P(X = k) = e^(−λ) λ^k / k!` from `n` i.i.d. observations.

Solution. Log-likelihood:

```
ℓ(λ) = Σᵢ (−λ + xᵢ log λ − log(xᵢ!)) = −nλ + log(λ) Σᵢ xᵢ + const
```

Derivative:

```
∂ℓ/∂λ = −n + (1/λ) Σᵢ xᵢ = 0
λ̂ = (1/n) Σᵢ xᵢ = x̄
```

Again the sample mean — Poisson mean equals λ.

Problem 3 — CLT check. You have a sample of `n = 64` observations with `x̄ = 100` and `s = 16`. Compute the 95% CI for the population mean.

Solution.

```
SE = s / √n = 16 / 8 = 2
CI = 100 ± 1.96 × 2 = [96.08, 103.92]
```

Problem 4 — Small-sample t. Same setup but `n = 9`. The 97.5 percentile of the `t(8)` distribution is about 2.306 (instead of 1.96 for Normal). Compute the CI.

Solution.

```
SE = 16 / 3 = 5.333
CI = 100 ± 2.306 × 5.333 = [87.7, 112.3]
```

Note how much wider the CI is for small `n` — both because `SE` scales with `1/√n` and because the critical value is larger.

Problem 5 — Interpret. A statistician reports "the 95% CI for average bus arrival delay is `[3.2, 5.6]` minutes." Your manager says "So there's a 95% chance the real delay is between 3.2 and 5.6 minutes?" Is the manager right? Why or why not?

Solution. No. The manager is stating a Bayesian credible interval, but the statistician reported a frequentist CI. The correct interpretation is: "If we repeated the sampling procedure many times, about 95% of the constructed intervals would contain the true mean delay." The *true mean* is a fixed number — either in this interval or not — so it does not have a probability. In practice, with a flat prior and large `n`, the numerical answer is often the same, but the interpretation differs.

Cross-References

- **Lesson 2.3** applies MLE machinery to hypothesis tests.
- **Lesson 2.5** derives OLS as the MLE of a linear regression under Normal errors.
- **Lesson 2.6** derives logistic regression as MLE over Bernoulli responses.
- **Module 3** uses MAP (L2/L1 regularisation) to stabilise high- dimensional models.
- **Module 6** uses MLE gradients as the core of RLHF-style fine-tuning.

Reflection

Write the log-likelihood for the simplest model you care about in your own work (number of clicks, revenue per user, call duration). Take the derivative by hand. Does the result agree with the sklearn or Polars function you would normally call? Almost always, yes. That sameness is the point of MLE: modern software is a thin wrapper around the derivation you just did.

Lesson 2.3 — Bootstrapping and Hypothesis Testing

Why This Matters

Suppose you roll out a new homepage and conversion rises from 4.5% to 4.9% over 50,000 visits. Is that a real improvement or just noise? An MLE gives you a point estimate. Lesson 2.2 gave you a formula-based confidence interval. But what if your statistic is something weird — the 95th percentile of session duration, or the Gini coefficient of revenue distribution — and no neat formula exists? What if your sample is small and the CLT shakes?

This lesson gives you two tools that work when theory alone isn't enough:

1. **The bootstrap** lets you estimate the sampling distribution of *any* statistic by resampling with replacement. It's one of the most powerful ideas in modern statistics.
2. **Hypothesis testing** frames your question as a decision: is the observed effect bigger than what random noise would produce? We cover p-values (and their widespread misinterpretation), permutation tests, power analysis, and multiple-testing corrections.

Together, these tools let you turn "that number looks bigger" into a defensible scientific claim.

Core Concepts — The Bootstrap

Efron's insight

In 1979 Bradley Efron asked a deceptively simple question: if the sampling distribution of a statistic is what we care about, and we can't draw new samples from the population, why not *resample from the sample we have*? Each resample is a best-effort simulation of "another draw from the population," and the distribution of the statistic across resamples approximates the true sampling distribution.

The algorithm:

1. Given a sample $x_1, \ldots, x_n$, draw n observations **with replacement** to produce a **bootstrap sample** $x_1^*, \ldots, x_n^*$.
2. Compute the statistic of interest on the bootstrap sample. Call this a **bootstrap replicate** $\hat{\theta}^*$.
3. Repeat steps 1–2 B times (typically `B = 1000` or `10_000`).
4. Use the distribution of $\hat{\theta}^{*1}, \hat{\theta}^{*2}, \ldots, \hat{\theta}^{*B}$ as a surrogate for the true sampling distribution of $\hat{\theta}$.

Why "with replacement"?

If you sampled without replacement, every bootstrap sample would equal the original sample, and you'd learn nothing. With replacement, some observations appear multiple times and some are missing — each bootstrap sample is a slightly different "alternate universe" draw from the same population.

On average, a bootstrap sample contains about `1 − 1/e ≈ 63.2%` of the unique observations from the original sample. The rest are duplicates.

Percentile confidence interval

The simplest bootstrap CI is the **percentile method**: sort the bootstrap replicates and take the `α/2` and `1 − α/2` quantiles.

For a 95% CI with `B = 1000`:

```
lower = sorted(replicates)[25]     (index for 2.5%)
upper = sorted(replicates)[975]    (index for 97.5%)
CI = [lower, upper]
```

No distributional assumptions required. No `(σ/√n)` formula. Just count.

BCa intervals (THEORY)

The percentile method is biased when the bootstrap distribution is skewed. The **bias-corrected and accelerated** (BCa) interval fixes this. The full derivation is beyond the scope of this lesson, but the idea:

1. Compute a **bias correction** z_0 measuring how far the median of the bootstrap distribution is from the original estimate.
2. Compute an **acceleration** $\hat{a}$ measuring how much the variance of the estimator changes with the parameter value (via jackknife).
3. Use z_0 and $\hat{a}$ to adjust the percentiles used for the interval.

```
α₁ = Φ(z₀ + (z₀ + z_(α/2))   / (1 − â × (z₀ + z_(α/2))))
α₂ = Φ(z₀ + (z₀ + z_(1−α/2)) / (1 − â × (z₀ + z_(1−α/2))))
```

BCa intervals are second-order accurate (error `O(1/n)` rather than `O(1/√n)`) and should be your default whenever you have enough compute. `scipy.stats.bootstrap(..., method='BCa')` gives them to you for free.

When the bootstrap fails

1. **Extrema.** The bootstrap cannot produce values outside the observed range. It underestimates uncertainty for maxima, minima, and order statistics near the tails.
2. **Heavy tails.** If the population has infinite variance, the bootstrap inherits the problem.
3. **Dependent data.** Standard bootstrap destroys time-series structure. Use a *block bootstrap* or a *stationary bootstrap* instead.
4. **Very small `n`.** Below about `n = 10`, the bootstrap distribution is too discrete to be useful.
5. **Heteroscedastic errors in regression.** Use the **wild bootstrap** instead.

Parametric vs non-parametric bootstrap

- **Non-parametric bootstrap** (what we just described) resamples the raw data. No model assumed.
- **Parametric bootstrap** fits a parametric model (e.g. a Normal distribution with MLE parameters), then draws new samples from *that model*. Useful when you trust the model family but need resamples.

Mathematical Foundations — Hypothesis Testing

The Neyman-Pearson framework

Hypothesis testing frames inference as a decision problem. You have:

- **Null hypothesis H_0.** The default, typically "no effect."
- **Alternative H_1.** The thing you'd believe if H_0 were false.
- **Test statistic `T`.** A function of the data whose distribution under H_0 is known (at least approximately).
- **Significance level α.** The probability of rejecting H_0 when it is actually true. Common choices: $\alpha = 0.05$ (social science), $\alpha = 0.01$, or $\alpha = 5 \times 10^{-7}$ (physics "five sigma").

You compute `T` from the data, compare it to its distribution under H_0, and compute a **p-value**: the probability, under H_0, of observing a test statistic at least as extreme as the one you saw.

p-values — what they really mean

> ⚠ **Pitfall — The single most common statistical mistake.** A p-value is **NOT** the probability that H_0 is true. It is **NOT** the probability that your finding is false. It is **NOT** `1 - P(H₁ is true)`. Do not say any of these things.

The correct interpretation:

> "The probability of observing data at least as extreme as ours, *given that H_0 is true*."

Notice the conditioning: `P(data | H₀)`. That's not the same as `P(H₀ | data)` — we're back to the prosecutor's fallacy from Lesson 2.1.

If $p \le \alpha$, we **reject** H_0. If $p > \alpha$, we **fail to reject** H_0. We *never* say "accept H_0" — absence of evidence is not evidence of absence.

Type I vs Type II errors

	H_0 true	H_0 false
Reject H_0	Type I error (prob = α)	Correct rejection (prob = $1-\beta$)
Fail to reject	Correct (prob = $1-\alpha$)	Type II error (prob = β)

- α = significance level = Type I error rate. You set this.

- **β** = Type II error rate. You don't set it directly, but you can design around it.
- **1 − β** = **power**. The probability of detecting a real effect.
- **Effect size** = the magnitude of the true difference you want to catch.

Relationship. For a two-sample test with equal sample sizes:

```
n (per group) ≈ 2 × ((z_(1−α/2) + z_(1−β)) × σ / δ)²
```

where `δ` is the minimum detectable effect, `σ` is the pooled standard deviation, and `z_p` is the standard normal `p`-th percentile.

Numerical example. You want to detect a SGD 10,000 increase in 4-room HDB price (`δ = 10_000`) with `σ = 80_000`, `α = 0.05`, power 0.80 (`β = 0.20`). Then `z_(0.975) ≈ 1.96` and `z_(0.80) ≈ 0.84`. So:

```
n ≈ 2 × ((1.96 + 0.84) × 80_000 / 10_000)²
  ≈ 2 × (2.8 × 8)²
  ≈ 2 × 501.8
  ≈ 1003 per group
```

You need about 1000 flats per group, or 2000 total, to detect a 10K difference with 80% power. If you only have 200 flats per group, you cannot reasonably expect to detect that effect — the experiment is *underpowered*.

The t-statistic

For a one-sample test of `H₀: μ = μ₀`:

```
t = (x̄ − μ₀) / (s / √n)
```

Under `H₀` and assuming Normal errors, `t` follows a Student's t-distribution with `n − 1` degrees of freedom. When `n` is large (`n > 30` is a common rule of thumb), `t` is nearly standard Normal.

Derivation intuition. The numerator is how far the sample mean is from the hypothesised value, measured in SGD. The denominator is the standard error of the sample mean, also in SGD. The ratio is *unitless*: how many standard errors away is our estimate?

Two-sample t-test

For comparing two independent samples:

```
t = (x̄_A − x̄_B) / √(s_A² / n_A + s_B² / n_B)     (Welch's)
```

Welch's t-test does not assume equal variances and is the default modern choice. Under `H₀: μ_A = μ_B`, `t` is approximately `t(ν)` where `ν` is the Welch-Satterthwaite approximation of degrees of freedom.

Permutation tests

A **permutation test** is an exact, assumption-free alternative to the t-test. Procedure:

1. Combine both groups into one pool of `n_A + n_B` observations.
2. Randomly permute and re-split into groups of the original sizes.
3. Compute the test statistic (e.g. the difference of means) on the permuted split.
4. Repeat many times. The p-value is the fraction of permutations whose statistic is at least as extreme as the observed one.

Under `H₀` (the two groups come from the same distribution), every permutation is equally likely, so the observed statistic should sit somewhere random in the permutation distribution. If it's in the extreme tail, that's evidence

against H_0.

Permutation tests are exact (no Normal approximation needed), work for any statistic, and make no distributional assumptions. They are the gold standard when you can afford the compute.

Multiple testing

If you test one hypothesis at `α = 0.05`, you have a 5% chance of a false positive. If you test 20 hypotheses at `α = 0.05`, you expect *one* false positive by chance, even if none of the 20 effects is real. This is the **multiple testing problem**.

Bonferroni correction (conservative). Divide `α` by the number of tests `m`:

```
α_adjusted = α / m
```

If you test 20 hypotheses and want FWER (family-wise error rate) below 5%, test each at `α = 0.05 / 20 = 0.0025`.

Bonferroni is simple but aggressive — it controls the probability of *any* false positive, which is often too strict.

Benjamini-Hochberg FDR (recommended in practice). Controls the expected proportion of false discoveries among rejections, not the family-wise rate. Procedure:

1. Sort p-values: `p_(1) ≤ p_(2) ≤ … ≤ p_(m)`.
2. Find the largest `k` such that `p_(k) ≤ (k / m) × q` where `q` is your desired FDR.
3. Reject all hypotheses with p-values `≤ p_(k)`.

BH FDR keeps the *proportion* of false discoveries below `q` while allowing more rejections than Bonferroni when many tests are significant.

The Kailash Engine — ExperimentTracker for A/B Tests

A well-run A/B test logs everything: sample sizes, effect size, p-value, CI, power, and any adjustments for multiple testing. `ExperimentTracker` is built for this.

```
from kailash_ml import ExperimentTracker
from scipy import stats
import numpy as np

group_a = np.array([...])   # control conversions: 0 or 1
group_b = np.array([...])   # treatment conversions

p_a = group_a.mean()
p_b = group_b.mean()
diff = p_b - p_a

# Bootstrap CI for the difference
rng = np.random.default_rng(42)
B = 10_000
diffs = np.empty(B)
for b in range(B):
    a_star = rng.choice(group_a, len(group_a), replace=True)
    b_star = rng.choice(group_b, len(group_b), replace=True)
    diffs[b] = b_star.mean() - a_star.mean()
ci_lo, ci_hi = np.quantile(diffs, [0.025, 0.975])

# Permutation test p-value
```

```
pool = np.concatenate([group_a, group_b])
obs = diff
perm_diffs = np.empty(B)
for b in range(B):
    rng.shuffle(pool)
    perm_diffs[b] = pool[len(group_a):].mean() - pool[:len(group_a)].mean()
p_val = float(np.mean(np.abs(perm_diffs) >= np.abs(obs)))

with ExperimentTracker().start_run(name="homepage_ab_test") as run:
    run.log_param("test_type", "permutation")
    run.log_param("alpha", 0.05)
    run.log_param("n_bootstrap", B)
    run.log_metric("p_a", float(p_a))
    run.log_metric("p_b", float(p_b))
    run.log_metric("diff", float(diff))
    run.log_metric("ci_lo", float(ci_lo))
    run.log_metric("ci_hi", float(ci_hi))
    run.log_metric("p_value", p_val)
```

Three months later, if someone asks "what was the CI on our homepage test?", the tracker has it. If someone wants to re-run with a different α, the parameters are right there.

Worked Example — Full A/B Analysis

Setup. 10,000 visitors split evenly between two homepage variants. Variant A has 451 conversions (4.51%). Variant B has 502 conversions (5.02%). Is this real?

Step 1: Point estimate.

```
p_a = 451 / 5000 = 0.0902         NO wait, we said 10k split evenly
```

Let me redo the setup: 5000 per group. Then `p_a = 451/5000 = 0.0902 = 9.02%`. That's high for a homepage. Let's make it realistic: 50_000 per group, 2255 conversions for A (4.51%) and 2510 for B (5.02%).

```
p_a = 2255/50000 = 0.0451
p_b = 2510/50000 = 0.0502
diff = 0.0051 (0.51 percentage points)
```

Step 2: Analytic CI via Normal approximation.

```
se_diff = √(p_a(1 − p_a)/n_a + p_b(1 − p_b)/n_b)
        = √(0.0451 × 0.9549 / 50000 + 0.0502 × 0.9498 / 50000)
        = √(8.61e−7 + 9.54e−7)
        = √1.815e−6
        ≈ 0.001348

CI = 0.0051 ± 1.96 × 0.001348 = [0.00246, 0.00774]
```

Both endpoints are positive, so 0 is *not* in the 95% CI. This suggests `H₀: diff = 0` would be rejected at `α = 0.05`.

Step 3: z-test p-value.

```
z = 0.0051 / 0.001348 ≈ 3.78
p = 2 × (1 − Φ(3.78)) ≈ 0.00016
```

Highly significant. The observed difference would occur under `H₀` less than 2 times in 10,000.

Step 4: Power analysis. What minimum effect could we have detected at 80% power with these sample sizes?

```
MDE ≈ (1.96 + 0.84) × se_diff ≈ 2.8 × 0.001348 ≈ 0.00378
```

We could reliably detect differences of about 0.38 percentage points or larger. The observed 0.51 exceeds this, so the test was well-powered.

Step 5: Interpret. Variant B converts about 0.51 percentage points better than A (95% CI [0.25, 0.77]). $p < 0.001$. With 50K per arm, this is a well-powered test that is very unlikely to be a fluke. **Recommendation:** roll out B. But remember the **effect size** matters as much as significance — a 0.51 percentage-point lift at $10 per conversion and 100K weekly visitors is roughly 510 × 2 × $10 = $10,200 per week. If the new variant costs $50K to build, ROI is about 5 weeks. Significance alone does not tell you that.

Try It Yourself

Problem 1 — Bootstrap median. A sample of 10 salaries (in thousands) is `[45, 48, 52, 55, 58, 60, 62, 65, 70, 95]`. Estimate the 95% CI for the *median* by bootstrap (conceptually — describe the procedure and give the expected CI width in words).

Solution. The procedure is: sample 10 values with replacement from this list, compute the median, repeat 10_000 times, take the 2.5 and 97.5 percentiles. The median of the original sample is `(58 + 60)/2 = 59`. The bootstrap distribution of the median will be concentrated near 59 but with significant spread because the median for small samples is a discrete statistic (it jumps between sample values). Expect a CI roughly like `[52, 65]`. Notice the CI is asymmetric — the outlier at 95 shifts some bootstrap medians upward.

Problem 2 — One-sample t-test. A new HDB valuation model predicts prices with an average error of SGD 25,000 on 25 flats, sample SD of errors = SGD 12,000. Is the average error significantly different from 0 at `α = 0.05`?

Solution.

```
t = 25_000 / (12_000 / √25) = 25_000 / 2400 ≈ 10.42
```

That's enormously large. Comparing to `t(24)` at `α = 0.05` two-sided (critical value ≈ 2.064), we reject H_0 decisively. p-value is essentially 0. The model has a systematic bias of about SGD 25K.

Problem 3 — Multiple testing. You run 15 A/B tests and obtain p-values `[0.002, 0.008, 0.01, 0.015, 0.03, 0.04, 0.06, 0.08, 0.1, 0.12, 0.2, 0.3, 0.4, 0.5, 0.7]`. Which do you reject at FDR `q = 0.05` (Benjamini-Hochberg)?

Solution. Sort (already sorted). Compute `(k/m) × q` for each:

```
k=1: 1/15 × 0.05 = 0.00333   p=0.002 ≤ 0.00333 ✓
k=2: 2/15 × 0.05 = 0.00667   p=0.008 > 0.00667
k=3: 3/15 × 0.05 = 0.01      p=0.01 ≤ 0.01    ✓
k=4: 4/15 × 0.05 = 0.01333   p=0.015 > 0.01333
k=5: 5/15 × 0.05 = 0.01667   p=0.03 > 0.01667
…
```

Largest `k` with `p_(k) ≤ (k/m) × q`: `k = 3`. Reject hypotheses with the three smallest p-values: `0.002, 0.008, 0.01`.

Under Bonferroni, `α_adj = 0.05/15 = 0.00333`, we would reject only `0.002`. FDR gives us two extra rejections while still controlling false discoveries.

Problem 4 — Power. You want to detect a 2% lift in conversion from a baseline of 10% with 90% power. What sample size do you need per group? (Assume `α = 0.05` two-sided.)

Solution.

```
p₁ = 0.10, p₂ = 0.12, δ = 0.02
σ² ≈ p × (1 − p), use pooled p = 0.11 → σ² ≈ 0.0979
z_(0.975) ≈ 1.96, z_(0.90) ≈ 1.28

n ≈ 2 × ((1.96 + 1.28)² × 0.0979) / 0.02²
  ≈ 2 × (10.498 × 0.0979) / 0.0004
  ≈ 2 × 1.028 / 0.0004
  ≈ 5140 per group
```

(Approximately — formulas vary slightly.) You need roughly 5000 per group. If you have 10K visitors a week, you're running a week-long experiment.

Problem 5 — Interpret a p-value. A colleague says, "The test returned p = 0.03. So there's only a 3% chance our result is wrong." What is the correct thing to say?

Solution. "There's a 3% chance of observing data this extreme if there were truly no effect." The 3% is `P(data | H₀)`, not `P(error)` and definitely not `P(H₀ | data)`. The colleague's phrasing is a version of the base-rate fallacy; you need a prior on `H₀` to compute `P(H₀ | data)`, which is what Bayesian analysis does.

Cross-References

- **Lesson 2.4** designs the A/B test from scratch and covers SRM.
- **Lesson 2.5** t-statistic reappears as the significance test on a regression coefficient.
- **Lesson 2.7** CUPED uses the bootstrap as a sanity check on variance reduction.
- **Module 3** cross-validation uses the same bootstrap principle to estimate generalisation error.

Reflection

Think of a claim from your last sprint review — "X improved by Y%." Was there a confidence interval on `Y`? If the interval crossed zero, the claim is statistically indistinguishable from "nothing changed." This is the single biggest reason decisions based on small-sample point estimates go wrong.

Lesson 2.4 — A/B Testing and Experiment Design

Why This Matters

Lessons 2.1–2.3 gave you the tools. Lesson 2.4 shows you how to use them in practice, before and during an experiment, so you don't end up with data that can't answer your question. The hardest part of A/B testing isn't the math — it's the *design*. A perfectly executed test of the wrong hypothesis is worse than useless; it's actively misleading.

You will learn:

1. The anatomy of a hypothesis: formulation, metric, randomisation.
2. A data-collection framework: **why / what / where / how / frequency**.
3. Common pitfalls: overlapping treatments, temporal bias, silos, aggregated data.
4. **Sample Ratio Mismatch (SRM)** detection — the single most important sanity check before you interpret any A/B test.
5. Clean DataOps architecture for reproducible experiments.

Core Concepts — Experiment Design

Hypothesis formulation

A good hypothesis has three parts:

1. **Change.** What are you doing differently? "Show a BOGO banner on the home page."
2. **Expected effect.** What is the predicted direction and magnitude of impact? "Average daily spend per customer increases by SGD 5."
3. **Metric.** How will you measure it? "Daily spend (SGD) per unique customer over 14 days."

Write it as one sentence: *"The BOGO banner increases daily spend per customer by at least SGD 5 over a 14-day period."* Now you have something falsifiable, measurable, and scoped.

Common mistakes:

- **Vague change.** "Make the homepage better." What does better mean? Which element changed?
- **Composite metric.** "Engagement" is not a metric; it's a cloud of metrics. Pick one primary metric (what you'll make the decision on) and secondary metrics (for monitoring and post-hoc analysis).
- **No effect size.** If you don't know what size of change matters, you can't power your experiment.

Randomisation

Randomisation is the one thing that separates experiments from observational analysis. By assigning subjects to treatment and control *randomly*, you ensure the two groups are statistically equivalent on every variable you measure *and* every variable you don't. Any difference in outcome can then be attributed to the treatment.

Things that break randomisation:

1. **Overlapping treatments.** A user sees both A and B in the same session. Exposure is not clean.
2. **Selection on behaviour.** "Show the new feature to users who opt in to beta." Opt-in users differ systematically from the rest.
3. **Carryover effects.** Users see variant A, then a week later switch to B. Behaviour under B is influenced by A.
4. **Temporal bias.** All of variant A runs on Monday and all of variant B on Tuesday. Day-of-week effects confound the comparison.

The fix is almost always **unit-level random assignment**: hash `user_id` → bucket, keep users in the same bucket throughout the experiment, run both variants in parallel for the full duration.

The data collection framework — why / what / where / how / frequency

Before you start an experiment, write down five things.

Why — hypotheses being tested.

- What business decision depends on this experiment?
- What's the **value impact** — dollars, users, retention?
- What are the **performance measures** (metrics)?

What — the wish list of ideal information.

- What data would you collect if time and budget were infinite?
- Which fields are critical vs nice-to-have?

- What's the budget / time constraint that forces the cut?

Where — sources of data.

- **Internal:** CRM, POS, web/mobile app events, finance systems.
- **External:** public datasets (data.gov.sg, Kaggle), third-party APIs, purchased data.
- Which sources are authoritative? Which are best-effort?

How — data capture mechanics.

- **At the start:** manual review, discovery, validation of definitions.
- **Continuous:** automated event streams, batch ETL, human- augmented annotation.
- **Real-time vs batch:** real-time for interventions, batch for reporting.

Frequency — how often?

- **Per transaction** vs **batch** vs **ad hoc**.
- **Collection frequency** (how often you ingest) vs **analysis frequency** (how often you compute results).
- **Duration:** long enough for statistical power, short enough to act.

This framework sounds bureaucratic, but every team that has been burned by bad data swears by it. A 30-minute conversation at the start prevents weeks of post-hoc confusion.

Common data collection problems

- **Silos.** Marketing owns ad impressions, product owns web events, finance owns revenue. Joining them is an archaeology project.
- **Red tape.** Approvals, data privacy reviews, DPO sign-offs. Start early.
- **Incomplete or inconsistent data.** Different teams use different definitions of "active user." Align before you measure.
- **Aggregated vs transaction-level.** Daily totals can hide individual patterns. Always prefer transaction-level when possible.
- **Leakage.** Future information sneaking into the training set. We'll cover this formally in Lesson 2.7.

Clean DataOps architecture

A production-grade experimentation pipeline looks like:

1. **Schema.** Every event type has a versioned schema with explicit types, nullability, and ranges.
2. **Transfer.** Events flow from source systems into a warehouse (e.g. via CDC, Kafka, or batch).
3. **Processing.** Event-level data is aggregated into experiment-ready metrics (per user, per day, per cohort).
4. **Connectors.** Experiment analysis tools (your code, `ExperimentTracker`) consume these metrics through a well-defined interface, never by reaching into the raw tables.

Kailash's `FeatureStore` and `ExperimentTracker` together implement this architecture out of the box. You get schema validation, versioning, and lineage for free — provided you use the engines instead of rolling your own ETL.

Sample Ratio Mismatch (SRM)

Suppose you designed a 50/50 A/B test. You expect roughly equal numbers of users in each arm. When the data comes in, arm A has 52,000 users and arm B has 48,000. Is this bad?

Yes. It's called a **Sample Ratio Mismatch** and it is *the* single most common experiment-ruining bug. Causes include:

- Bot filtering applied to one arm but not the other.
- Redirect bug that drops some users before they reach the assignment code.
- Population filter applied post-hoc to one arm.
- Logging loss on one variant.
- Eligibility checks run on different data in each arm.

Detection is a chi-squared goodness-of-fit test. Under the expected split `p_A : p_B = 0.5 : 0.5`:

```
expected_A = n × 0.5
expected_B = n × 0.5

chi² = (observed_A − expected_A)² / expected_A
     + (observed_B − expected_B)² / expected_B
```

With 1 degree of freedom and `α = 0.001` (very strict for SRM), the critical value is about 10.83.

Numerical example. `n = 100_000`, observed `A = 52_000`, `B = 48_000`, expected `A = B = 50_000`.

```
chi² = (52_000 − 50_000)² / 50_000 + (48_000 − 50_000)² / 50_000
     = 4_000_000 / 50_000 + 4_000_000 / 50_000
     = 80 + 80
     = 160
```

That's way above 10.83. SRM is confirmed — your experiment is broken. Do not interpret the treatment effect. Fix the assignment pipeline and re-run.

> ⚠ **Pitfall — Rationalising SRM.** "It's only a 4% difference, probably fine." No. A 4% difference at `n = 100K` is overwhelming evidence of a broken pipeline. The threshold for SRM panic is a p-value below 0.001, *not* below 0.05.

The Kailash Engine — ExperimentTracker for Design

`ExperimentTracker` has explicit support for experiment design: you record the hypothesis, the power analysis, the randomisation scheme, and the SRM check as metadata on the experiment, *before* you look at any outcome data.

```python
from kailash_ml import ExperimentTracker
from scipy import stats

tracker = ExperimentTracker()
with tracker.start_run(name="homepage_bogo_banner") as run:
    # Design-time parameters
    run.log_param("hypothesis",
                  "BOGO banner increases 14d spend by >= SGD 5/customer")
    run.log_param("primary_metric", "spend_14d_per_customer")
    run.log_param("randomization", "user_id_hash_bucket")
    run.log_param("alpha", 0.05)
    run.log_param("power", 0.80)
    run.log_param("mde_sgd", 5.0)
    run.log_param("baseline_std_sgd", 35.0)
    # Computed sample size for Welch's t-test:
    n_per_arm = 2 * ((1.96 + 0.84) * 35 / 5) ** 2
    run.log_param("n_per_arm_planned", int(n_per_arm))
    run.log_param("duration_days", 14)
```

```
# After launch, log the actual assignment and SRM check:
observed_a, observed_b = 52_000, 48_000
expected = (observed_a + observed_b) / 2
chi2 = ((observed_a - expected)**2 + (observed_b - expected)**2) / expected
p_srm = 1 - stats.chi2.cdf(chi2, df=1)
run.log_metric("srm_chi2", float(chi2))
run.log_metric("srm_p_value", float(p_srm))
if p_srm < 0.001:
    run.log_param("srm_status", "FAIL — stop analysis")
else:
    run.log_param("srm_status", "PASS")
```

The magic of logging the design *before* the results is that you cannot be tempted to rationalise after the fact. The hypothesis, the MDE, the power, and the SRM threshold are all committed before you see an outcome number.

Worked Example — Designing the BOGO Banner Test

Business context. A retailer wants to know whether adding a BOGO banner to the home page increases 14-day customer spend by enough to cover the production cost.

Step 1 — Hypothesis.

> *"The BOGO banner increases 14-day average spend per customer by at least SGD 5 compared to the current home page."*

Step 2 — Primary and secondary metrics.

- Primary: 14-day spend per unique customer (SGD).
- Secondary: click-through rate on banner, session duration, return-visit rate.

Step 3 — Power analysis. Historical data: baseline spend per customer = SGD 80, SD = 35. We want to detect `δ = 5` with `α = 0.05` and power 0.80.

```
n_per_arm ≈ 2 × ((1.96 + 0.84) × 35 / 5)² ≈ 2 × 19.6² ≈ 768
```

Round up to 1000 per arm for safety margin. With about 500 visitors per day per arm, we need 2 days minimum — but run for 14 to capture weekly seasonality.

Step 4 — Randomisation. Hash user_id modulo 2. Assign bucket 0 to control, bucket 1 to treatment. Stable across sessions.

Step 5 — Launch and monitor SRM. After day 1, pull assignment counts. If `|n_A - n_B| / n > 0.01` or the SRM chi-squared p-value < 0.001, halt and investigate.

Step 6 — Analyse. At day 14, compute the two-sample Welch's t-test and a bootstrap CI on the difference. Also log the secondary metrics for context.

Step 7 — Decide. If the 95% CI excludes 0 and the lower bound exceeds the break-even point (say SGD 2), roll out. If the CI straddles 0, fail to reject the null and do not roll out — but *also* report the point estimate and CI so stakeholders can see how uncertain the result was.

Try It Yourself

Problem 1 — SRM check. You ran a 50/50 test with 200,000 total users, observing 101,500 in arm A and 98,500 in arm B. Is this SRM?

Solution.

```
expected = 100_000
chi² = (1500² + 1500²) / 100_000 = 4_500_000 / 100_000 = 45
```

45 is way above the 10.83 threshold at $p < 0.001$. SRM confirmed. Investigate.

Problem 2 — Underpowered test. You have 100 users per arm and want to detect a 1% lift on a 10% baseline conversion. Can you do it at `α = 0.05`, power 0.80?

Solution. Pooled `p ≈ 0.105`, `σ² ≈ 0.094`.

```
n ≈ 2 × (2.8² × 0.094) / 0.01² ≈ 2 × 0.737 / 0.0001 ≈ 14_740
```

You need roughly 15,000 per arm, not 100. The test is hopelessly underpowered. Either run much longer or redesign the hypothesis.

Problem 3 — Hypothesis critique. Rewrite this bad hypothesis: "We want to see if the new checkout flow is better."

Solution.

> *"The new checkout flow increases completion rate by at least 0.5 percentage points compared to the current flow, over a 30-day period among all desktop users."*

Change, metric, direction, effect size, scope — all explicit.

Problem 4 — Which metric? You're testing a recommendation carousel. Which should be the primary metric: click-through rate, add-to-cart, purchase, or 7-day revenue per customer?

Solution. 7-day revenue per customer. CTR captures only the first action; purchases can be dragged around by the carousel without increasing overall revenue. Revenue captures the ultimate business goal.

Problem 5 — Design the frequency. Your team wants to rerun a homepage A/B test every two weeks. Given `n = 5000` visitors per day, baseline 5% conversion, and a 0.5-percentage-point MDE, is 14 days enough?

Solution.

```
Required n per arm ≈ 2 × (2.8² × 0.05 × 0.95) / 0.005²
                   ≈ 2 × 0.372 / 0.000025
                   ≈ 29_760
```

So ~60K users total, or 12 days at 5000/day. 14 days works with a small margin. If you added a weekend effect buffer, you might push to 21 days.

Cross-References

- **Lesson 2.3** supplies the statistical machinery.
- **Lesson 2.7** explains how CUPED reduces the sample size required for a given power.
- **Module 4** uses experiment tracking for hyperparameter sweeps, where the "variants" are model configurations.
- **Module 6** uses the same design discipline for prompt optimisation and RLHF reward model selection.

Reflection

Take the last experiment your team ran. Answer: was the hypothesis pre-registered? Was there a power analysis? Was randomisation at the user level or session level? Was SRM checked? If any answer is "no," next time fix it *before* running the test.

Lesson 2.5 — Linear Regression

Why This Matters

Linear regression is the first model most data scientists learn, and — if you treat it seriously — the most important one. Nearly every more complex model is a generalisation of it: logistic regression, GLMs, neural networks (a linear layer is just OLS per neuron), ridge, lasso, even the attention mechanism in transformers is a soft version of linear projection.

More importantly, regression is how you *explain* a prediction. A coefficient has a direction, a magnitude, and a significance. You can say "holding all else equal, a one-unit increase in X is associated with a $\hat{\beta}$ change in Y, and we are 95% sure that effect is between `a` and `b`." No tree, no deep network, and no AutoML pipeline gives you that.

This lesson derives OLS, the t-statistic on a coefficient, R-squared, and the F-statistic from first principles, using matrix algebra where it simplifies things and keeping things scalar where it does not.

Core Concepts

The linear model

Given n observations with p predictors each, the model is:

$$y_i = \beta_0 + \beta_1 \times x_{i1} + \beta_2 \times x_{i2} + \dots + \beta_p \times x_{ip} + \varepsilon_i$$

where ε_i is random noise. The **assumptions** are:

1. **Linearity.** The mean of y is linear in the parameters.
2. **Independence.** Errors are independent across observations.
3. **Homoscedasticity.** Errors have constant variance σ^2.
4. **Normality.** Errors are Normally distributed (needed only for inference, not for point estimation).
5. **No perfect multicollinearity.** No predictor is an exact linear combination of the others.

In **matrix form**:

$$y = X\beta + \varepsilon$$

where y is an $n \times 1$ vector, X is an $n \times (p+1)$ design matrix (first column is all 1s for the intercept), β is a $(p+1) \times 1$ parameter vector, and ε is an $n \times 1$ error vector.

Ordinary Least Squares

OLS finds the β that minimises the sum of squared residuals:

$$S(\beta) = \Sigma_i (y_i - \hat{y}_i)^2 = (y - X\beta)^T (y - X\beta)$$

Expand:

$$S(\beta) = y^T y - 2\beta^T X^T y + \beta^T X^T X\beta$$

Differentiate with respect to β and set to zero:

$$\partial S/\partial\beta = -2X^T y + 2X^T X\beta = 0$$
$$X^T X\beta = X^T y$$

Solve (assuming $X^T X$ is invertible):

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

This is the most important formula in statistics. Let's understand each piece:

- $X^T y$ is a $(p+1) \times 1$ vector: the cross-product of each predictor with the response.
- $X^T X$ is a $(p+1) \times (p+1)$ matrix: the "covariance structure" of the predictors (unscaled).
- $(X^T X)^{-1}$ undoes the predictor covariance so each $\hat{\beta}_j$ is the *unique* contribution of predictor `j` after accounting for all others.

For a simple regression (one predictor, intercept):

$$\hat{\beta}_1 = \mathrm{Cov}(x, y) / \mathrm{Var}(x)$$
$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \times \bar{x}$$

OLS as MLE under Normal errors

If $\varepsilon_i \sim N(0, \sigma^2)$ independently, then $y_i \sim N(x_i^T\beta, \sigma^2)$ and the log-likelihood is:

$$\ell(\beta, \sigma^2) = -(n/2) \log(2\pi\sigma^2) - (1/(2\sigma^2)) \times \Sigma_i (y_i - x_i^T\beta)^2$$

Maximising over β is equivalent to *minimising* $\Sigma_i (y_i - x_i^T\beta)^2$ — which is exactly OLS. So OLS = MLE under Normal errors. Beautiful.

Interpreting coefficients

- **Sign.** The direction of the relationship. Positive β = X and Y move together; negative = opposite.
- **Magnitude.** The size of the effect, in the original units of `y` per unit of `x`. A coefficient of 5000 on `sqm` means "each additional square metre is associated with SGD 5000 more price, on average, holding other predictors fixed."
- **Ceteris paribus.** Every coefficient is "all else equal." This is the multivariate version of adjusting for confounders.

Non-linear extensions in a linear framework

Linear regression is linear *in the parameters*, not in the predictors. You can add:

- **Polynomial terms:** x, x^2, x^3. Captures curvature.
- **Interaction terms:** $x_1 \times x_2$. Captures effects that depend on another variable.
- **Log transforms:** `log(y)` or `log(x)`. Captures multiplicative effects: a unit increase in `log(x)` corresponds to multiplying `x` by `e`.

Interpretation of log-linear. If $\log(y) = \beta_0 + \beta_1 x$, then a one-unit increase in `x` multiplies `y` by $e^{\beta_1} \approx 1 + \beta_1$ for small β_1. For example, $\hat{\beta}_1 = 0.05$ means "y increases by about 5% per unit of x."

Categorical encoding

For a categorical predictor with `k` levels, create `k − 1` dummy variables. The omitted level is the **base** (or reference) category. The intercept then represents the mean for the base, and each dummy's coefficient represents the *difference* from the base.

Example: flat type ∈ {3-room, 4-room, 5-room}. Let 3-room be the base. Then:

$\hat{y} = \beta_0 + \beta_4room \times I(4room) + \beta_5room \times I(5room) + \ldots$

β_0 is the average 3-room price (all else equal). `β_4room` is how much more (or less) a 4-room flat costs than a 3-room flat, holding all else equal.

> ⚠ **Pitfall — Dummy variable trap.** If you include *all* `k` dummies plus an intercept, X^TX is singular (the dummies sum to 1, which equals the intercept column). $\hat{\beta}$ cannot be computed. Drop one dummy; `pd.get_dummies(..., drop_first=True)` handles this automatically in pandas, and Polars' `to_dummies` combined with dropping a column does the same.

Mathematical Foundations — Inference

Standard error of a coefficient

Under the assumptions, the variance-covariance matrix of $\hat{\beta}$ is:

$Var(\hat{\beta}) = \sigma^2 \times (X^TX)^{-1}$

We don't know σ^2, so we estimate it from the residuals. The **unbiased estimator** of σ^2 is:

$\hat{\sigma}^2 = (1 / (n - p - 1)) \times \Sigma_i (y_i - \hat{y}_i)^2$

The divisor is `n − p − 1` (sample size minus number of fitted parameters including intercept) — the residual degrees of freedom. This is Bessel's correction generalised.

The standard error of coefficient `j` is:

$SE(\hat{\beta}_j) = \sqrt{(\hat{\sigma}^2 \times [(X^TX)^{-1}]_jj)}$

where $[(X^TX)^{-1}]_jj$ is the `j`-th diagonal entry of the inverse.

t-statistic

To test $H_0: \beta_j = 0$ (the predictor has no effect after accounting for others), use:

$t_j = \hat{\beta}_j / SE(\hat{\beta}_j)$

Under H_0 and Normal errors, $t_j \sim t(n - p - 1)$. For `n` large, cutoffs:

- $|t| > 1.64 \rightarrow p < 0.10$ (90% confidence)
- $|t| > 1.96 \rightarrow p < 0.05$ (95% confidence)
- $|t| > 2.58 \rightarrow p < 0.01$ (99% confidence)

This is the *same* t-statistic from Lesson 2.3, applied to a regression coefficient instead of a sample mean. The conceptual unity is the entire point.

R-squared

R^2 is the proportion of variance in `y` explained by the model:

```
SS_tot = Σᵢ (yᵢ − ȳ)²     (total variance)
SS_res = Σᵢ (yᵢ − ŷᵢ)²    (residual variance)
SS_reg = SS_tot − SS_res  (explained variance)

R² = 1 − SS_res / SS_tot = SS_reg / SS_tot
```

$R^2 = 0$ means the model explains nothing beyond the sample mean. $R^2 = 1$ means it fits perfectly. In the social sciences, $R^2 = 0.3$ is often considered good; in physics, lower than 0.9 is embarrassing.

> ⚠ **Pitfall — R^2 always rises with more predictors.** Adding random features can only increase R^2. Use **adjusted R^2**:
>
> ```
> R²_adj = 1 − (SS_res / (n − p − 1)) / (SS_tot / (n − 1))
>  = 1 − (1 − R²) × (n − 1) / (n − p − 1)
> ```
>
> which penalises additional parameters.

F-statistic

The **model F-test** tests $H_0: \beta_1 = \beta_2 = \ldots = \beta_p = 0$ (all slopes are zero; the model is no better than the intercept-only model). The statistic is:

```
F = (SS_reg / p) / (SS_res / (n − p − 1))
  = (R² / p) / ((1 − R²) / (n − p − 1))
```

Under H_0, $F \sim F(p, n - p - 1)$. Large F ($p < 0.05$) rejects H_0: at least one coefficient is non-zero.

The F-test is for **overall model significance**. Individual t-tests are for **coefficient significance**. Use both: F tells you the model is useful, and t tells you which predictors matter.

Numerical example — HDB with two predictors

Suppose $n = 5$. Predictors: floor area (sqm) and age (years). Response: price (SGD 000s).

i	area	age	price
1	60	10	420
2	70	5	490
3	80	15	510
4	90	20	540
5	100	25	580

Design matrix `X` (with intercept column):

```
X = [[1, 60, 10],
     [1, 70, 5],
     [1, 80, 15],
     [1, 90, 20],
     [1, 100, 25]]
y = [420, 490, 510, 540, 580]
```

Compute $\hat{\beta} = (X^T X)^{-1} X^T y$ (you would do this in NumPy, but the point is that a closed-form solution exists). Numerical answer (rounded): $\hat{\beta} \approx [318, 3.0, -2.5]$. Interpretation:

- Baseline (area=0, age=0): SGD 318K — meaningless extrapolation, but a necessary intercept.
- Each additional square metre: +SGD 3,000.
- Each additional year of age: –SGD 2,500.

Fit:

```
ŷ = [318 + 3×60 − 2.5×10, 318 + 3×70 − 2.5×5, …]
  = [473, 515.5, 520.5, 538, 555.5]
Residuals e = y − ŷ = [−53, −25.5, −10.5, 2, 24.5]
SS_res ≈ 2809 + 650 + 110 + 4 + 600 ≈ 4173
ȳ = 508, SS_tot = (420−508)² + … + (580−508)² = 7744 + 324 + 4 + 1024 + 5184 = 14_280
R² = 1 − 4173/14_280 ≈ 0.708
```

So ~71% of variance explained. Not great for HDB prices (there are many more confounders — location, lease, storey), but consistent with a deliberately tiny example.

The Kailash Engine — TrainingPipeline + ModelVisualizer

kailash-ml's `TrainingPipeline` wraps OLS fitting with experiment-tracking and visualisation. Conceptually:

```python
from kailash_ml import TrainingPipeline, ModelVisualizer, ExperimentTracker
import polars as pl

from shared import MLFPDataLoader

loader = MLFPDataLoader()
hdb = loader.load("mlfp01", "hdb_resale.parquet")

features = hdb.select([
    "floor_area_sqm",
    "remaining_lease_years",
    "storey_mid",
    "town",
    "flat_type",
]).to_pandas()  # for the estimator boundary; kailash-ml exposes polars internally
target = hdb["resale_price"].to_numpy()

pipeline = TrainingPipeline(task="regression", estimator="ols")
pipeline.fit(features, target)

print(pipeline.summary())     # coefficients, SE, t, p, R², F
ModelVisualizer().coefficient_plot(pipeline)
```

The `summary()` method produces a regression table with every inferential statistic we derived by hand: $\hat{\beta}$, `SE`, `t`, `p-value`, `95% CI`, R^2, adjusted R^2, F statistic and its p-value, and residual diagnostics.

Worked Example — Predicting HDB Prices

Target: 4-room resale price. Predictors: floor area, storey, remaining lease, distance to CBD, town (one-hot).

Expected coefficient signs (your *prior*):

- `floor_area_sqm`: positive. Bigger flats cost more.
- `storey_mid`: slightly positive. Higher floors have better views.

- `remaining_lease_years`: positive. Longer leases are worth more.
- `distance_to_cbd_km`: negative. Closer to Raffles costs more.
- `town` dummies: vary. Central towns positive, outer towns negative, with "Ang Mo Kio" as base.

After fitting ($n \approx 50_000$), a plausible table:

Predictor	$\hat{\beta}$	SE	t	p
intercept	280_000	5_000	56.0	< 0.001
floor_area_sqm	3_200	60	53.3	< 0.001
storey_mid	2_500	150	16.7	< 0.001
remaining_lease_years	900	80	11.25	< 0.001
distance_to_cbd_km	–15_000	300	–50.0	< 0.001
town[BISHAN]	55_000	2_500	22.0	< 0.001
town[TOA PAYOH]	40_000	2_300	17.4	< 0.001
… (other towns)	…	…	…	…

$R^2 = 0.84$, adjusted $R^2 = 0.839$, $F = 3200$ ($p < 0.001$). Every coefficient is significant. Signs match priors. The model is well-specified enough that we can start using it for valuations — *with the caveat* that R^2 on training data isn't generalisation error; we'll need Module 3's cross-validation to verify.

Try It Yourself

Problem 1 — Simple regression by hand. Given `x = [1, 2, 3, 4, 5]`, `y = [2, 4, 5, 4, 5]`, compute $\hat{\beta}_0$ and $\hat{\beta}_1$.

Solution.

```
x̄ = 3, ȳ = 4
Σ (xᵢ − x̄)(yᵢ − ȳ) = (−2)(−2) + (−1)(0) + (0)(1) + (1)(0) + (2)(1) = 6
Σ (xᵢ − x̄)² = 4 + 1 + 0 + 1 + 4 = 10
β̂₁ = 6/10 = 0.6
β̂₀ = 4 − 0.6 × 3 = 2.2
ŷ = 2.2 + 0.6x
```

Problem 2 — R^2 for the above.

Solution.

```
ŷ = [2.8, 3.4, 4.0, 4.6, 5.2]
Residuals = [−0.8, 0.6, 1.0, −0.6, −0.2]
SS_res = 0.64 + 0.36 + 1.0 + 0.36 + 0.04 = 2.4
SS_tot = (2−4)² + (4−4)² + (5−4)² + (4−4)² + (5−4)² = 4 + 0 + 1 + 0 + 1 = 6
R² = 1 − 2.4/6 = 0.6
```

60% of variance explained.

Problem 3 — t-statistic. A regression reports $\hat{\beta}_1$ `= 1500`, `SE = 300`, `n − p − 1 = 200`. Is the coefficient significant at `α = 0.05`?

Solution.

```
t = 1500 / 300 = 5.0
```

$t = 5$ with $df = 200$ is far beyond any conventional cutoff. $p < 0.001$. Yes, highly significant.

Problem 4 — Interaction term. You fit `price = β₀ + β₁ × area + β₂ × central + β₃ × (area × central)`. What does β_3 represent?

Solution. β_3 is the *additional* increase in price per square metre for flats in central locations, beyond the baseline increase β_1. In other words, the effect of area is β_1 for non-central flats and $\beta_1 + \beta_3$ for central flats. If $\beta_3 > 0$, square metres are more valuable in central locations (as you'd expect).

Problem 5 — Categorical base. You encode flat_type with 3-room as base. $\hat{\beta}$`_4room = 50_000`, $\hat{\beta}$`_5room = 90_000`. Interpret.

Solution. Holding all else equal, a 4-room flat costs SGD 50K more than a 3-room, and a 5-room flat costs SGD 90K more than a 3-room. The 5-room premium over 4-room is `90K − 50K = 40K`. If the standard errors and t-stats support it, every pair is significantly different from every other.

Cross-References

- **Lesson 2.2** justifies OLS as MLE.
- **Lesson 2.3** supplies the t-distribution for coefficient inference.
- **Lesson 2.6** extends linear to logistic regression via the log-odds link.
- **Lesson 2.7** uses regression for CUPED's variance reduction and for DiD identification.
- **Module 3** adds regularisation (ridge, lasso) and cross- validated model selection.
- **Module 5** uses linear regression as a probing tool for LLM hidden states.

Reflection

Pick any model in your organisation and ask: what are the coefficients, and what's the sign and magnitude of each? If no one can answer, the model is a black box even to its authors. Every production model deserves a regression-style explanation next to it, even if the model itself is something fancier.

Lesson 2.6 — Logistic Regression and Classification Foundations

Why This Matters

Linear regression is for continuous outcomes. Half the problems you'll face are binary: clicks or no clicks, churn or stay, default or pay, positive or negative. You *could* try to fit a linear model to `y ∈ {0, 1}`, but the predictions quickly slip out of `[0, 1]` and the coefficients become nonsensical. The right tool is **logistic regression**.

Logistic regression is the most important classical classifier and the foundation of the sigmoid activation function in neural networks. Every machine-learning class that eventually arrives at deep learning started here — and if you understand logistic regression deeply, the leap to binary cross-entropy loss in a neural network is one line.

This lesson also covers **ANOVA**, the natural multi-group generalisation of the two-sample t-test. ANOVA is used everywhere in experimental science; in ML, it appears when you compare more than two variants (A/B/C/D tests) or multiple subgroups.

Core Concepts — From Linear to Logistic

The linearity problem for binary responses

Suppose you fit $y = \beta_0 + \beta_1 \times x + \varepsilon$ where $y \in \{0, 1\}$ (e.g. $y = 1$ if a customer clicks). Two things go wrong:

1. **Predictions outside [0, 1].** Linear functions have no upper or lower bound. For some x, $\hat{y}$ will be negative or greater than 1 — which is meaningless as a probability.
2. **Heteroscedastic errors.** The variance of a Bernoulli is $p(1 - p)$, which depends on the mean. OLS assumes constant variance; this assumption is violated.

The fix: transform the linear predictor through a function that maps $(-\infty, +\infty)$ to $(0, 1)$. That function is the **sigmoid** (also called the logistic function):

$$\sigma(z) = 1 / (1 + e^{-z})$$

The sigmoid is bounded in $(0, 1)$, smooth, and monotonic. Its inverse is the **logit**:

$$\text{logit}(p) = \log(p / (1 - p))$$

$p / (1 - p)$ is the **odds ratio**: if $p = 0.8$, odds = 4 (i.e. 4 to 1 in favour). $\log(\text{odds})$ is the **log-odds**.

Logistic regression model

The logistic regression model is:

$$P(y = 1 \mid x) = \sigma(\beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p)$$

equivalently:

$$\text{logit}(P(y = 1 \mid x)) = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p$$

The log-odds are linear in the predictors. The probability is a non-linear (sigmoid) transformation of the linear predictor.

Odds ratio interpretation

From the model:

$$\log(p / (1 - p)) = \beta_0 + \beta_1 x_1 + \ldots$$

A one-unit increase in x_1 (holding others fixed) changes the log-odds by β_1. Exponentiating, this multiplies the odds by e^{β_1}. So:

- $\beta_1 = 0 \rightarrow e^0 = 1 \rightarrow$ odds unchanged.
- $\beta_1 = 0.693 \rightarrow e^{0.693} = 2 \rightarrow$ odds **double**.
- $\beta_1 = -0.693 \rightarrow e^{(-0.693)} = 0.5 \rightarrow$ odds **halve**.

Numerical example. Suppose a churn model has $\beta_{discount} = -0.5$. Then each additional 10% discount multiplies churn odds by $e^{(-0.5)} \approx 0.607$, i.e. cuts churn odds by about 39%.

Mathematical Foundations — Maximum Likelihood for Logistic Regression

The likelihood

Assuming $y_i \in \{0, 1\}$ and $p_i = \sigma(x_i^T\beta)$:

$$L(\beta) = \prod_i p_i^{y_i} \times (1 - p_i)^{(1 - y_i)}$$

Log-likelihood:

$$\ell(\beta) = \Sigma_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Substituting $p_i = \sigma(x_i^T\beta)$ and using $\log(\sigma(z)) = -\log(1 + e^{(-z)})$:

$$\ell(\beta) = \Sigma_i [y_i \times x_i^T\beta - \log(1 + e^{(x_i^T\beta)})]$$

This is the **negative binary cross-entropy** (up to a sign), which is also the loss function used in deep learning for binary classification. Same function, two names.

Gradient and why there's no closed form

Take the derivative with respect to β:

$$\partial\ell/\partial\beta = \Sigma_i (y_i - p_i) \times x_i$$

Beautiful. The residual in the probability space is $y_i - p_i$; the gradient is the sum of residuals weighted by the feature vector.

Unfortunately, setting this to zero does *not* yield a closed form because p_i depends non-linearly on β. We solve it by iterative methods — typically **iteratively reweighted least squares** (IRLS) or Newton-Raphson. In practice, sklearn and kailash-ml use `liblinear` or L-BFGS solvers that converge in a handful of iterations.

Sigmoid derivative (useful identity)

$$\sigma'(z) = \sigma(z) \times (1 - \sigma(z))$$

Derivation: write $\sigma(z) = 1 / (1 + e^{(-z)})$ and differentiate using the chain rule:

$$\begin{aligned}\sigma'(z) &= (-1) \times (1 + e^{(-z)})^{(-2)} \times (-e^{(-z)}) \\ &= e^{(-z)} / (1 + e^{(-z)})^2 \\ &= [1 / (1 + e^{(-z)})] \times [e^{(-z)} / (1 + e^{(-z)})] \\ &= \sigma(z) \times (1 - \sigma(z))\end{aligned}$$

This identity is the entire reason sigmoid is so nice to work with — its derivative is a product of its output and one minus its output, no extra variables.

Multiclass extensions

For $K > 2$ classes, there are three common approaches:

1. **One-vs-rest (OvR).** Train K binary classifiers, each "class k" vs "not class k." Predict the class with highest score. Simple but not calibrated.

2. **One-vs-one (OvO).** Train $K(K-1)/2$ pairwise classifiers. Majority vote. More accurate but $O(K^2)$.

3. **Multinomial (softmax) regression.** Generalise the sigmoid to the softmax:

   $$P(y = k \mid x) = e^{(\beta_k^T x)} / \Sigma_j e^{(\beta_j^T x)}$$

   Fit by maximising the multinomial log-likelihood. This is what neural networks use for multi-class heads.

Evaluation preview

Logistic regression outputs probabilities, not classes. To classify, you threshold: $\hat{y} = 1$ if $\hat{p} \geq 0.5$, else 0. You'll see in Module 3 that this threshold is a tunable hyperparameter, and the right setting depends on the cost of false positives vs false negatives. For now, think of logistic regression as a **probability model** first and a classifier second.

Metrics you'll meet in Module 3:

- **Accuracy** = fraction correct.
- **Precision** = `TP / (TP + FP)`.
- **Recall (sensitivity)** = `TP / (TP + FN)`.
- **F1** = harmonic mean of precision and recall.
- **ROC-AUC** = area under the receiver-operating characteristic curve, measuring ranking quality.
- **Log-loss** = $-\ell(\beta)$ on held-out data; smaller is better.

Core Concepts — ANOVA (one-way)

Why ANOVA?

A two-sample t-test compares two group means. With three or more groups you could do all pairwise t-tests, but the multiple-testing problem kicks in: three tests at $\alpha = 0.05$ give an overall false positive rate of about $1 - 0.95^3 \approx 14\%$.

ANOVA (Analysis of Variance) performs a single test of the joint hypothesis $H_0: \mu_1 = \mu_2 = \ldots = \mu_k$ (all group means are equal) against the alternative that at least one differs.

The F-statistic decomposition

Total variance in the outcome can be split:

```
SS_total = SS_between + SS_within
```

- $SS_{between} = \Sigma_g\, n_g\, (\bar{x}_g - \bar{x})^2$ — variance explained by group membership.
- $SS_{within} = \Sigma_g\, \Sigma_i\, (x_{i,g} - \bar{x}_g)^2$ — variance inside groups.

Divide each by its degrees of freedom:

```
MS_between = SS_between / (k - 1)
MS_within  = SS_within  / (N - k)
```

where k is the number of groups and N is the total sample size.

The F-statistic is:

```
F = MS_between / MS_within
```

Under H_0, $F \sim F(k-1, N-k)$. Large F ($p < \alpha$) rejects H_0: at least one group mean differs.

Intuition. If the between-group differences are large relative to the within-group noise, something is happening. If within-group noise dominates, group membership isn't informative.

Numerical example — HDB across flat types

Three groups: 3-room, 4-room, 5-room. Summary:

Group	n	x̄	s
3-room	100	400_000	60_000
4-room	100	540_000	80_000
5-room	100	680_000	90_000

Overall mean: `x̄ ≈ 540_000`.

```
SS_between = 100 × (400 − 540)² × 10^6
           + 100 × (540 − 540)² × 10^6
           + 100 × (680 − 540)² × 10^6
           = 100 × 19_600 × 10^6 + 0 + 100 × 19_600 × 10^6
           = 3.92e12
df_between = 3 − 1 = 2
MS_between = 1.96e12

SS_within = 99 × 60_000² + 99 × 80_000² + 99 × 90_000²
          = 99 × (3.6e9 + 6.4e9 + 8.1e9)
          = 99 × 1.81e10 ≈ 1.79e12
df_within = 300 − 3 = 297
MS_within ≈ 6.03e9

F = 1.96e12 / 6.03e9 ≈ 325
```

`F ≈ 325` with df `(2, 297)` — vastly beyond any reasonable critical value. We reject `H₀`. At least one flat-type mean differs (which — surprise — is all of them).

Post-hoc tests

ANOVA only tells you "somewhere, means differ." To find *which* pairs differ, run post-hoc tests with corrections:

- **Tukey's Honestly Significant Difference (HSD).** Tests all pairwise differences controlling the family-wise error rate. Default for most one-way ANOVAs.
- **Bonferroni pairwise.** Simpler but more conservative.
- **Scheffé.** Controls all possible contrasts, not just pairs. Most conservative.
- **Dunnett's.** Compares each group to a single control.

For our HDB example, all three pairs (3 vs 4, 3 vs 5, 4 vs 5) would be highly significant under Tukey's HSD.

ANOVA vs regression

ANOVA is a special case of regression with only categorical predictors. You can always refit a one-way ANOVA as a regression with dummy variables for groups (with one group as base), and the F-statistic from the regression equals the ANOVA F. In modern practice, most statisticians just run regressions and use ANOVA as a reporting frame.

The Kailash Engine — TrainingPipeline for Classification

```
from kailash_ml import TrainingPipeline, ModelVisualizer

pipeline = TrainingPipeline(task="classification", estimator="logistic")
pipeline.fit(X_train, y_train)

print(pipeline.summary())
```

```
# Coefficient    β̈       SE    z     p      exp(β̈)
# tenure_months  -0.04  0.003  -13   <.001   0.961
# discount_pct   -0.02  0.002  -10   <.001   0.980
# has_complaint  +1.20  0.100  +12   <.001   3.32
# …

ModelVisualizer().coefficient_plot(pipeline, as_odds_ratio=True)
```

The `exp(β̂)` column is the odds ratio. A value of 3.32 on "has_complaint" means customers with complaints have 3.32 times the odds of churning, all else equal.

Worked Example — Employee Attrition

Setup. You have HR data on 14_999 employees (roles, salaries, tenure, promotion history, attrition flag). Goal: predict `attrition ∈ {0, 1}` and interpret the top drivers.

Model. Logistic regression on standardised predictors. After fitting:

Predictor	β̂	OR	Interpretation
years_since_last_promotion	+0.35	1.42	Each additional year → 42% more churn odds
monthly_hours	+0.20	1.22	10 extra hours/mo → 22% more churn odds
promoted_last_2y	–0.60	0.55	Promotion → 45% less churn odds
compensation_pct_of_band	–0.45	0.64	Well-paid employees → 36% less churn odds
dept_sales	+0.30	1.35	Sales dept has 35% more churn odds than baseline

Interpretation for the CEO:

> "Three levers matter most for retention: make sure high performers are promoted within two years, keep monthly hours reasonable, and keep pay at or above band midpoint. The single biggest risk factor is going 2+ years without a promotion; it increases churn odds by 42% per extra year. The model has AUROC 0.83 on held-out data."

Try It Yourself

Problem 1 — Odds interpretation. A logistic regression reports `β̂_age = 0.02` with $p < 0.001$. Interpret the odds ratio.

Solution. `e^0.02 ≈ 1.0202`. Each additional year of age increases the odds of the positive class by about 2%, holding others fixed. Small but significant at large `n`.

Problem 2 — Sigmoid. Compute `σ(0)`, `σ(2)`, `σ(-2)`.

Solution.

```
σ(0) = 1 / (1 + e^0) = 1/2 = 0.5
σ(2) = 1 / (1 + e^(-2)) = 1 / (1 + 0.1353) ≈ 0.881
σ(-2) = 1 / (1 + e^2) = 1 / (1 + 7.389) ≈ 0.119
```

Symmetric around 0.

Problem 3 — Multiclass. You have three classes. Which multiclass scheme should you use if: (a) you need calibrated probabilities; (b) you're using a tree-based model?

Solution. (a) Multinomial (softmax) logistic regression — gives proper probabilities that sum to 1. (b) OvR is usually fine for trees, since tree-based calibration is already suspect.

Problem 4 — ANOVA. You run a 4-variant homepage test. Is the right tool ANOVA or pairwise t-tests?

Solution. ANOVA first to test "any difference." If significant, follow with Tukey HSD to find which pairs differ. Pairwise t-tests without correction inflate the false positive rate.

Problem 5 — ANOVA F. `MS_between = 800`, `MS_within = 40`. Compute F and describe the result.

Solution. `F = 20`. Enormously large — the effect is much bigger than noise. Depending on df, p is essentially 0.

Cross-References

- **Lesson 2.2** MLE — the same procedure drives logistic fit.
- **Lesson 2.5** linear regression — logistic is its sibling.
- **Module 3** calibration, threshold tuning, ROC curves.
- **Module 4** deep learning — sigmoid is the binary output activation; softmax generalises to multiclass.
- **Module 6** RLHF — the preference model is essentially a binary logistic regression over "preferred vs rejected" pairs.

Reflection

Find a binary classification model in your org. Ask: what are the top 5 coefficients (in log-odds, not some opaque feature importance score)? Do the signs agree with domain intuition? If not, either the model or your intuition is wrong — and you should find out which.

Lesson 2.7 — CUPED and Causal Inference

Why This Matters

You've seen how to design, run, and analyse an A/B test. But two problems remain:

1. **A/B tests are expensive.** To detect a 0.5-percentage-point lift in conversion, you may need tens of thousands of users per arm running for weeks.
2. **Randomisation isn't always possible.** You can't A/B test a government policy, a city-wide rollout, or a macro shock — the "randomisation" is done by history, not by you.

This lesson gives you two tools:

1. **CUPED (Controlled-experiment Using Pre-Experiment Data),** a variance-reduction technique that can cut required sample sizes by 50% or more. It is the single most impactful modern A/B testing technique, used by Microsoft, Netflix, Airbnb, and essentially every top tech company.
2. **Difference-in-Differences (DiD),** a causal inference method that works on observational data when you have pre/post data for a treated and a control group. Classic application: evaluating a policy change.

Both rest on the same core idea: **use pre-experiment data to reduce the role of chance in your estimate**.

Core Concepts — CUPED

The idea

Suppose you are testing whether a new homepage increases `Y` = spend per user. Without CUPED, you compare `Y_treatment` to `Y_control`. The variance of the estimated difference is `Var(Y_t) / n + Var(Y_c) / n`.

Key observation. Most of the variability in `Y` is not caused by the treatment — it's caused by the user's pre-existing behaviour. Some users spend a lot no matter what; some spend little no matter what. If we can *subtract out* the predictable part of `Y` using pre-experiment behaviour, we reduce the noise without touching the treatment effect.

The adjustment

Let `X_pre` be a pre-experiment covariate (e.g. the user's spend in the 14 days *before* the experiment started). Define the CUPED-adjusted outcome:

```
Y_adj = Y − θ × (X_pre − E[X_pre])
```

where `θ` is chosen to minimise `Var(Y_adj)`.

Deriving the optimal θ

```
Var(Y_adj) = Var(Y − θ X_pre + θ E[X_pre])
           = Var(Y − θ X_pre)                 (E[X_pre] is a constant)
           = Var(Y) − 2θ × Cov(Y, X_pre) + θ² × Var(X_pre)
```

Differentiate with respect to θ and set to zero:

```
−2 × Cov(Y, X_pre) + 2θ × Var(X_pre) = 0
θ* = Cov(Y, X_pre) / Var(X_pre)
```

This is *exactly* the OLS regression slope of `Y` on `X_pre`. CUPED is regression-adjusted estimation in disguise.

Deriving the variance reduction

Plug `θ*` back into the variance formula:

```
Var(Y_adj) = Var(Y) − 2 × (Cov(Y, X_pre)² / Var(X_pre))
                   + (Cov(Y, X_pre)² / Var(X_pre)² ) × Var(X_pre)
           = Var(Y) − 2 × Cov(Y, X_pre)² / Var(X_pre)
                   + Cov(Y, X_pre)² / Var(X_pre)
           = Var(Y) − Cov(Y, X_pre)² / Var(X_pre)
```

Using the definition of correlation `ρ = Cov(Y, X_pre) / (σ_Y × σ_X)`:

```
Cov(Y, X_pre)² / Var(X_pre) = ρ² × σ_Y² × σ_X² / σ_X² = ρ² × σ_Y²
```

So:

```
Var(Y_adj) = σ_Y² − ρ² σ_Y² = σ_Y² × (1 − ρ²)
```

Key result:

```
Var(Y_adj) = Var(Y) × (1 − ρ²)
```

That's it. If the correlation between pre- and post-experiment metrics is `ρ = 0.7`, variance drops by `1 − 0.49 = 0.51`: a **51% reduction**. If `ρ = 0.9`, variance drops by 81%.

Sample size multiplier

Required sample size scales with variance. So:

```
n_CUPED / n_raw = 1 − ρ²
```

ρ	Variance reduction	Sample size multiplier
0.3	9%	1.10
0.5	25%	1.33
0.7	49%	1.96
0.8	64%	2.78
0.9	81%	5.26
0.95	90%	10.0

With `ρ = 0.8`, your 50K-user experiment becomes an 18K-user experiment at the same power. Run it three times as fast.

Caveats

- **The covariate must be pre-experiment.** If `X` is measured during the experiment, it can be affected by the treatment, and subtracting it biases the estimate.
- **θ should be estimated from both arms pooled** (not from one arm), to avoid bias.
- **CUPED reduces variance, not bias.** If your experiment is broken (SRM, leakage, bad randomisation), CUPED won't save you.
- **CUPED is regression.** If you understand linear regression with a single pre-experiment covariate, you understand CUPED.

Core Concepts — Causal Inference Basics

Rubin's potential outcomes

For each unit `i`, there are two potential outcomes:

- $Y_i(1)$ = outcome if `i` is treated.
- $Y_i(0)$ = outcome if `i` is not treated.

The **individual treatment effect** is:

$$\tau_i = Y_i(1) - Y_i(0)$$

The Fundamental Problem of Causal Inference: you observe at most one of $Y_i(1)$ and $Y_i(0)$. The other is the **counterfactual** — what *would have* happened. You can never directly observe an individual treatment effect.

The way out is to estimate averages:

- **ATE (Average Treatment Effect):** `E[Y(1) − Y(0)]` across the population.
- **ATT (Average Treatment Effect on the Treated):** `E[Y(1) − Y(0) | treated]`.
- **CATE (Conditional ATE):** `E[Y(1) − Y(0) | X = x]`.

Randomised experiments estimate ATE directly because the treated and untreated groups are (on average) identical. For observational data, you need assumptions about how people select into treatment.

Mathematical Foundations — Difference-in-Differences

The setup

You have two groups (treated, control) observed at two time points (pre, post). The treatment happens between pre and post, but *only for the treated group*. You want the causal effect of the treatment.

Naive approaches fail:

- **Post-only difference:** compare `Y_treat_post` to `Y_control_post`. Fails if the groups weren't comparable before treatment.
- **Treated pre/post:** compare `Y_treat_post` to `Y_treat_pre`. Fails if time trends affect both groups.

DiD combines both:

```
ATT = (Y_treat_post - Y_treat_pre) - (Y_control_post - Y_control_pre)
```

In words: take the change in the treated group and subtract the change in the control group. The difference is the causal effect — assuming the control's change is a valid proxy for what the treated group *would have* experienced without treatment.

The parallel trends assumption

DiD identifies the ATT only if, in the counterfactual world without treatment, the treated and control groups would have experienced the *same change* over time. Formally:

```
E[Y(0)_post - Y(0)_pre | treated] = E[Y(0)_post - Y(0)_pre | control]
```

This is untestable (we never observe the counterfactual) but we can check it empirically with **pre-trends**: if the two groups were evolving in parallel for several periods *before* treatment, the assumption is more credible.

Regression form

DiD can be written as a regression with three variables:

$$Y_{it} = \beta_0 + \beta_1 \times D_i + \beta_2 \times T_t + \delta \times (D_i \times T_t) + \varepsilon_{it}$$

- $D_i = 1$ if unit i is in the treated group.
- $T_t = 1$ if period t is post-treatment.
- $D_i \times T_t = 1$ only for treated units in the post period.

The coefficient δ on the interaction *is* the DiD estimate. The standard error comes from the usual OLS formula (with clustered standard errors if you have many observations per unit).

Placebo tests

The credibility of DiD rests entirely on the parallel-trends assumption. You can stress-test it:

1. **Fake treatment time.** Pretend treatment happened one period earlier (when nothing actually changed). If DiD finds a significant "effect," parallel trends fails.
2. **Fake treatment group.** Pretend a third group, similar to the control, was treated. Should find no effect.
3. **Pre-trend regression.** Regress the outcome on time within the pre-treatment period, separately for treated and control. Slopes should be similar.

Numerical example — Singapore ABSD cooling measures

In December 2021, Singapore raised the Additional Buyer's Stamp Duty (ABSD) for investment properties. To estimate the policy's causal effect on HDB resale prices, we need a control group that was *not* affected.

- **Treated:** non-owner-occupier (investment) purchases.
- **Control:** first-time buyers (exempt from ABSD hike).

Pre-period: Jan–Nov 2021. Post-period: Jan–Nov 2022.

Hypothetical means (SGD, simplified):

	Pre	Post	Change
Treated (investment)	560_000	585_000	+25_000
Control (first-time)	520_000	560_000	+40_000

```
DiD = (585_000 - 560_000) - (560_000 - 520_000)
    = 25_000 - 40_000
    = -15_000
```

Investment-segment prices grew SGD 15K *less* than they would have under the counterfactual (proxied by first-time buyers). That's the causal effect of the ABSD hike. Statistically significant if the standard error supports it.

Parallel trends check. Plot both groups' monthly mean prices from 2019 through mid-2021. Are the slopes similar? If yes, assumption holds. If the treated group was already decelerating before the policy, DiD over-estimates the effect.

The Kailash Engine — ExperimentTracker + Regression

```python
from kailash_ml import ExperimentTracker, TrainingPipeline
import numpy as np
import polars as pl

# ── CUPED ──────────────────────────────────────────────────────
# X_pre: pre-experiment metric, Y: outcome, T: treatment indicator
theta = np.cov(Y, X_pre, ddof=1)[0, 1] / np.var(X_pre, ddof=1)
Y_adj = Y - theta * (X_pre - X_pre.mean())

rho = float(np.corrcoef(Y, X_pre)[0, 1])
var_reduction = 1 - rho ** 2

with ExperimentTracker().start_run(name="homepage_test_cuped") as run:
    run.log_param("covariate", "spend_pre_14d")
    run.log_metric("rho", rho)
    run.log_metric("variance_reduction", var_reduction)
    run.log_metric("theta", theta)

    # raw estimate
    diff_raw = Y[T == 1].mean() - Y[T == 0].mean()
    se_raw = (Y[T == 1].var(ddof=1)/sum(T==1)
              + Y[T == 0].var(ddof=1)/sum(T==0)) ** 0.5
    run.log_metric("diff_raw", diff_raw)
    run.log_metric("se_raw", se_raw)

    # cuped estimate
    diff_cuped = Y_adj[T == 1].mean() - Y_adj[T == 0].mean()
    se_cuped = (Y_adj[T == 1].var(ddof=1)/sum(T==1)
```

```
                + Y_adj[T == 0].var(ddof=1)/sum(T==0)) ** 0.5
    run.log_metric("diff_cuped", diff_cuped)
    run.log_metric("se_cuped", se_cuped)

# — DiD ————————————————————————————————————————————————
# df has columns: price, treated (0/1), post (0/1)
pipeline = TrainingPipeline(task="regression", estimator="ols")
pipeline.fit(df[["treated", "post", "treated_x_post"]].to_pandas(),
             df["price"].to_numpy())
print(pipeline.summary())
# δ (coefficient on treated_x_post) is the DiD estimate.
```

Worked Example — CUPED on a Conversion Test

Suppose Y has variance 100, and a pre-experiment covariate X has correlation `ρ = 0.8` with Y. The raw required sample size for 80% power at `α = 0.05`, MDE = 2, is:

```
n_raw = 2 × (2.8 × 10 / 2)² = 2 × 196 = 392 per arm
```

With CUPED, adjusted variance is `100 × (1 − 0.64) = 36`, so:

```
n_cuped = 2 × (2.8 × 6 / 2)² = 2 × 70.56 = 141.12 → 142 per arm
```

You need **142 instead of 392** per arm — a 64% reduction, run the experiment nearly 3x faster.

Try It Yourself

Problem 1 — CUPED math. `ρ = 0.6`. What is the variance reduction and sample size multiplier?

Solution. `1 − 0.36 = 0.64` variance remaining. So sample size multiplier is `1/0.64 = 1.5625`. Variance reduction is 36%. Sample size needed is 64% of raw.

Problem 2 — θ estimation. `Cov(Y, X_pre) = 30`, `Var(X_pre) = 25`. Compute `θ*`.

Solution. `θ* = 30 / 25 = 1.2`.

Problem 3 — Parallel trends. You look at monthly averages 2018–2020 for treated and control groups. Treated is growing at 2% per quarter, control at 1.5% per quarter. Does parallel trends hold?

Solution. Not cleanly — there's a 0.5% gap in growth rates. You'd do a formal pre-trend regression and check if the difference is statistically significant. If yes, DiD is biased; consider Synthetic Control or an event-study model instead.

Problem 4 — DiD estimate. `Y_treat_pre = 100`, `Y_treat_post = 130`, `Y_control_pre = 90`, `Y_control_post = 110`. Compute the DiD.

Solution.

```
DiD = (130 − 100) − (110 − 90) = 30 − 20 = 10
```

Treatment added 10 units beyond the underlying time trend.

Problem 5 — When DiD fails. A marketing campaign launches simultaneously with a city-wide subway-closure crisis. What happens to your DiD estimate?

Solution. If the crisis affects only the treated city (or asymmetrically), the control is no longer a valid proxy and parallel trends fails. DiD confounds the subway effect with the campaign effect. Fix: find a better control (another city unaffected by the crisis) or wait for the post-crisis data.

Cross-References

- **Lesson 2.3 and 2.4** supply the A/B testing framework CUPED improves.
- **Lesson 2.5** linear regression — CUPED is OLS with one covariate, and DiD is OLS with an interaction.
- **Module 3** cross-validated regression estimates the CUPED θ out-of-sample for extra robustness.
- **Module 6** causal reasoning underlies RLHF reward assignment — a preference is a causal claim.

Reflection

Pick a question your team answers with an A/B test. Is there a pre-experiment metric that correlates with the outcome? If yes, your next experiment can be 30–70% faster with CUPED. That alone is worth the cost of this lesson.

Lesson 2.8 — Capstone: Statistical Analysis Project

Why This Matters

Everything so far has been taught piece by piece. Real projects never look like that. You get a vague question, some messy data, a deadline, and an audience that doesn't care about p-values. You have to pull together every tool from Module 2 — probability, estimation, testing, regression, logistic regression, and causal inference — and produce a *useful* answer.

This final lesson walks through a complete end-to-end project on Singapore HDB data. The narrative is: **"What drives HDB resale prices, and are price changes between years driven by flat characteristics or by market dynamics?"**

The Full Pipeline

Step 1 — Load and explore

```
import polars as pl
from kailash_ml import DataExplorer
from shared import MLFPDataLoader

loader = MLFPDataLoader()
hdb = loader.load("mlfp01", "hdb_resale.parquet")

DataExplorer().explore(hdb, target="resale_price")
```

The explorer prints schema, summary stats, missing counts, and target distribution. Use it to catch typos, out-of-range values, and imbalanced categories before modelling.

Step 2 — Feature engineering

Temporal features (month, quarter, year), geographic features (distance to CBD, proximity to nearest MRT), and interaction terms (area × central) are the first round. See Module 3 for a systematic approach; here we prototype.

```
from kailash_ml import FeatureEngineer

engineer = FeatureEngineer()
hdb_feat = engineer.add_temporal(hdb, date_col="month")
hdb_feat = engineer.add_distance_to_point(
    hdb_feat, lat_col="lat", lon_col="lon",
    point_lat=1.2833, point_lon=103.8500, name="dist_to_raffles_km"
)
```

Step 3 — Hypothesis

Write three hypotheses, each testable:

1. *"Floor area is the single strongest predictor of price, controlling for lease and location."*
2. *"The 2021 cooling measures reduced investment-segment growth by at least SGD 10K."*
3. *"Remaining lease affects price non-linearly: the last 30 years of lease lose value faster than the middle 30."*

Step 4 — Tests

- Hypothesis 1: fit a multivariate OLS, look at the t-statistic on `floor_area_sqm` and the partial R^2 contribution.
- Hypothesis 2: DiD with treated = investment, control = first-time buyers.
- Hypothesis 3: fit a regression with `remaining_lease` and `remaining_lease²`; check if the squared term is significant.

Step 5 — Model

```
from kailash_ml import TrainingPipeline

pipeline = TrainingPipeline(task="regression", estimator="ols")
pipeline.fit(hdb_feat.select([
    "floor_area_sqm", "remaining_lease_years", "dist_to_raffles_km",
    "storey_mid", "year", "flat_type", "town"
]).to_pandas(),
    hdb_feat["resale_price"].to_numpy())

print(pipeline.summary())
```

Step 6 — Interpret

Write a two-paragraph narrative for a non-technical audience:

> "Floor area is the biggest driver of HDB resale prices: each additional square metre adds about SGD 3,200, all else equal. Location matters almost as much — being 1 km closer to Raffles is worth SGD 15K. Remaining lease has a non-linear effect: flats with less than 30 years of lease lose value sharply, consistent with CPF loan-eligibility rules.
>
> "The December 2021 ABSD hike reduced investment-segment price growth by SGD 18K relative to the first-time-buyer control group (95% CI [SGD 12K, SGD 24K], $p < 0.001$). The policy had a measurable cooling effect on the targeted segment without dampening first-time-buyer prices."

Step 7 — Present

A good final report has:

1. **Executive summary** (3 bullets). What you found and what to do.
2. **Chart 1:** coefficient plot with 95% CIs, sorted by magnitude. Labels in plain English.
3. **Chart 2:** DiD event-study plot showing both groups' means over time, vertical line at treatment date.
4. **Methodology appendix.** Data sources, assumptions, checks run (SRM, pre-trends, residual diagnostics).

Kailash Engines — Full Integration

```
from kailash_ml import (
    DataExplorer, PreprocessingPipeline, FeatureEngineer,
    FeatureStore, FeatureSchema, TrainingPipeline,
    ExperimentTracker, ModelVisualizer
)

# 1. Schema
schema = FeatureSchema({
    "resale_price": {"dtype": "float", "description": "SGD"},
    "floor_area_sqm": {"dtype": "float", "min": 20, "max": 300},
    "remaining_lease_years": {"dtype": "float", "min": 0, "max": 99},
    "dist_to_raffles_km": {"dtype": "float", "min": 0},
    "town": {"dtype": "categorical"},
})

# 2. Feature store
store = FeatureStore()
store.register_schema("mlfp02_hdb", schema)
store.ingest("mlfp02_hdb", hdb_feat)

# 3. Retrieve with as-of (prevents leakage)
features_2024 = store.get_features("mlfp02_hdb", as_of="2024-12-31")

# 4. Train
pipeline = TrainingPipeline(task="regression", estimator="ols")
pipeline.fit(features_2024.drop("resale_price").to_pandas(),
             features_2024["resale_price"].to_numpy())

# 5. Log everything
with ExperimentTracker().start_run(name="mlfp02_capstone") as run:
    run.log_param("model", "ols")
    run.log_param("n", len(features_2024))
    run.log_metric("r_squared", pipeline.r_squared_)
    run.log_metric("adjusted_r_squared", pipeline.adj_r_squared_)
    run.log_metric("f_statistic", pipeline.f_stat_)
    run.log_artifact("coefficient_plot",
                     ModelVisualizer().coefficient_plot(pipeline))
```

Every step is logged, versioned, and lineage-traced. A year from now, you (or a new teammate) can rerun the exact analysis.

Cross-References

- **Module 3** takes this capstone and adds proper cross-validation, regularisation, and model selection.

- **Module 4** introduces unsupervised methods (clustering, dimensionality reduction) as complements to the supervised tools we used here.
- **Module 5** adds LLM-based synthesis — a good final report can be drafted by an agent using the experiment tracker's metadata as context.
- **Module 6** adds alignment and governance — every model that goes to production needs a documented sign-off.

Reflection

Look at your current top project. Would it pass a Module 2 audit? Specifically:

1. Is there a pre-registered hypothesis?
2. Is there a CI on the main estimate?
3. Is there a check for common failure modes (SRM, leakage, multiple testing)?
4. Is the final answer framed causally, and if so, is the assumption explicit (randomisation, parallel trends, selection on observables)?

If any answer is "no," you now know what to do about it. That's the goal of Module 2: graduate from "running stats functions" to *statistical thinking*.

Module 2 Summary — Three Layers

Foundations (everyone).

- Probability is the language of uncertainty. Bayes' theorem updates beliefs when new evidence arrives.
- Population parameters (Greek) vs sample statistics (Latin).
- Confidence intervals are properties of the *procedure*, not of individual intervals.
- A/B tests need hypotheses, randomisation, power analysis, and SRM checks.
- Linear regression has four moving parts: coefficient direction, coefficient magnitude, coefficient significance (t-statistic), and overall model fit (R^2, F-test).
- Logistic regression is linear regression on log-odds, solved by MLE instead of closed-form OLS.

Theory (recommended).

- Maximum likelihood picks the parameter that makes the observed data most likely; MAP adds a prior.
- Fisher information and the Cramér-Rao bound tell you how precise your estimator can be.
- Bootstrap resamples with replacement to estimate sampling distributions when formulas don't exist.
- CUPED reduces variance by `(1 - ρ²)` via regression on a pre-experiment covariate.
- DiD identifies causal effects under the parallel trends assumption.
- ANOVA is regression with categorical predictors; the F-test generalises the two-sample t-test to multiple groups.

Advanced (optional).

- BCa bootstrap, wild bootstrap, block bootstrap.
- Bernstein-von Mises for prior-posterior convergence.
- Double ML and causal forests for heterogeneous treatment effects.
- Bayesian A/B testing (posterior probability that B > A).
- SUTVA violations and interference in network experiments.
- Permutation and conformal prediction.

Engines You Now Own

- **ExperimentTracker** — log every parameter and metric.
- **FeatureEngineer** — generate temporal, interaction, and polynomial features.
- **FeatureStore** — store features with schema, versioning, and point-in-time correctness.
- **TrainingPipeline** — fit linear and logistic regressions with full inferential output.
- **ModelVisualizer** — coefficient plots, residual diagnostics, posterior overlays.
- **DataExplorer** — from Module 1, used to sanity-check every new dataset before modelling.

What's Next

Module 3 takes this foundation and builds the production ML pipeline on top: cross-validation, regularisation, gradient boosting, explainability (SHAP), hyperparameter search, model registries, and deployment. Everything you learned in Module 2 — from the t-statistic to CUPED — continues to apply, now inside a full MLOps workflow.

> "Point-in-time correctness is non-negotiable." — carry this with you into Module 3 and every production model you ever build.

See you in Module 3: Supervised ML — Theory to Production.

Appendix A — Formula Reference

This appendix collects every formula in Module 2 in one place for quick lookup. Each formula is accompanied by a one-line reminder of what the symbols mean and a pointer back to the lesson where it was derived.

Probability and Distributions (Lesson 2.1)

```
Complement:              P(A') = 1 − P(A)
Union:                   P(A ∪ B) = P(A) + P(B) − P(A ∩ B)
Conditional:             P(A | B) = P(A ∩ B) / P(B)
Multiplication:          P(A ∩ B) = P(A) × P(B | A)
Independence:            P(A ∩ B) = P(A) × P(B)
Law of total prob:       P(B) = Σᵢ P(B | Aᵢ) × P(Aᵢ)
Bayes' theorem:          P(A | B) = P(B | A) × P(A) / P(B)
Expected value:          E[X] = Σᵢ pᵢ × xᵢ
Variance (form 1):       Var(X) = E[(X − μ)²]
Variance (form 2):       Var(X) = E[X²] − (E[X])²
Covariance:              Cov(X, Y) = E[(X − μ_X)(Y − μ_Y)]
Correlation:             ρ = Cov(X, Y) / (σ_X × σ_Y)
Normal density:          f(x) = (1/√(2π σ²)) × exp(−(x−μ)²/(2σ²))
Binomial pmf:            P(X = k) = C(n, k) × p^k × (1−p)^(n−k)
Poisson pmf:             P(X = k) = e^(−λ) × λ^k / k!
Exponential density:     f(x) = λ × e^(−λx),   x ≥ 0
```

Estimation and Inference (Lesson 2.2)

```
Population mean:         μ = E[X]
Population variance:     σ² = E[(X − μ)²]
Sample mean:             x̄ = (1/n) Σᵢ xᵢ
Sample variance:         s² = (1/(n−1)) Σᵢ (xᵢ − x̄)²
Standard error of mean:  SE(x̄) = σ / √n      (or s/√n when σ unknown)
Log-likelihood:          ℓ(θ) = Σᵢ log f(xᵢ; θ)
Normal MLE mean:         μ̂ = x̄
Normal MLE variance:     σ̂² = (1/n) Σᵢ (xᵢ − x̄)²
Bernoulli MLE:           p̂ = k / n
```

```
Fisher information:      I(θ) = −E[∂²ℓ/∂θ²]
Cramér-Rao bound:        Var(θ̂) ≥ 1 / (n × I(θ))
MAP estimator:           θ̂_MAP = argmax_θ (ℓ(θ) + log P(θ))
Normal-Normal posterior: 1/σₙ² = 1/σ₀² + n/σ²
                         μₙ = σₙ² × (μ₀/σ₀² + n×x̄/σ²)
95% CI (Normal):         x̄ ± 1.96 × (s / √n)
```

Hypothesis Testing (Lesson 2.3)

```
One-sample t:            t = (x̄ − μ₀) / (s / √n)     df = n − 1
Two-sample t (Welch):    t = (x̄_A − x̄_B) / √(s_A²/n_A + s_B²/n_B)
z-test (proportions):    z = (p̂_A − p̂_B) / SE_diff
Bonferroni:              α_adj = α / m
Benjamini-Hochberg:      reject p_(k) s.t. k = max{k : p_(k) ≤ (k/m) × q}
Bootstrap percentile CI: [quantile(θ̂*, α/2), quantile(θ̂*, 1−α/2)]
MDE (two-sample):        δ ≈ (z_(1−α/2) + z_(1−β)) × σ × √(2/n)
n per group:             n ≈ 2 × ((z_(1−α/2) + z_(1−β)) × σ / δ)²
```

A/B Testing (Lesson 2.4)

```
Chi-squared SRM:         χ² = Σ_g (observed_g − expected_g)² / expected_g
                         df = k − 1 (k = number of arms)
SRM threshold:           reject pipeline if p < 0.001
```

Linear Regression (Lesson 2.5)

```
Model (matrix):          y = Xβ + ε
OLS estimator:           β̂ = (Xᵀ X)⁻¹ Xᵀ y
Residuals:               e = y − Xβ̂
Residual variance:       σ̂² = eᵀe / (n − p − 1)
Var-Cov of β̂:           Var(β̂) = σ̂² × (Xᵀ X)⁻¹
Standard error:          SE(β̂ⱼ) = √(σ̂² × [(XᵀX)⁻¹]_jj)
Coefficient t-stat:      t_j = β̂_j / SE(β̂_j)     df = n − p − 1
SS total:                SS_tot = Σᵢ (yᵢ − ȳ)²
SS residual:             SS_res = Σᵢ (yᵢ − ŷᵢ)²
SS regression:           SS_reg = SS_tot − SS_res
R-squared:               R² = 1 − SS_res / SS_tot
Adjusted R²:             R²_adj = 1 − (1 − R²) × (n − 1)/(n − p − 1)
F-statistic:             F = (SS_reg / p) / (SS_res / (n − p − 1))
                         df = (p, n − p − 1)
```

Logistic Regression and ANOVA (Lesson 2.6)

```
Sigmoid:                 σ(z) = 1 / (1 + e^(−z))
Sigmoid derivative:      σ'(z) = σ(z) × (1 − σ(z))
Logit (log-odds):        logit(p) = log(p / (1 − p))
Model:                   logit(P(y=1 | x)) = xᵀβ
Log-likelihood:          ℓ(β) = Σᵢ [yᵢ × xᵢᵀβ − log(1 + e^(xᵢᵀβ))]
Gradient:                ∂ℓ/∂β = Σᵢ (yᵢ − pᵢ) × xᵢ
Odds ratio:              OR = e^β
Softmax (multiclass):    P(y = k | x) = e^(β_kᵀ x) / Σⱼ e^(β_jᵀ x)

ANOVA F:                 F = MS_between / MS_within
                         MS_between = SS_between / (k − 1)
                         MS_within  = SS_within  / (N − k)
SS_between:              Σ_g n_g (x̄_g − x̄)²
SS_within:               Σ_g Σ_i (xᵢ_g − x̄_g)²
```

CUPED and Causal Inference (Lesson 2.7)

```
CUPED adjustment:        Y_adj = Y − θ × (X_pre − E[X_pre])
Optimal theta:           θ* = Cov(Y, X_pre) / Var(X_pre)
CUPED variance:          Var(Y_adj) = Var(Y) × (1 − ρ²)
Sample size ratio:       n_CUPED / n_raw = 1 − ρ²

Potential outcomes:      τᵢ = Yᵢ(1) − Yᵢ(0)
ATE:                     E[Y(1) − Y(0)]
ATT:                     E[Y(1) − Y(0) | treated]
DiD estimator:           ATT = (Y_T,post − Y_T,pre) − (Y_C,post − Y_C,pre)
DiD regression form:     Y_it = β₀ + β₁ D_i + β₂ T_t + δ (D_i × T_t) + ε
                         δ is the DiD estimate
```

Appendix B — Glossary of Statistical Terms

Alpha (α). The significance level — the probability of a Type I error (rejecting H_0 when H_0 is true). Typical values are 0.05, 0.01, and, in high-stakes fields, smaller.

Alternative hypothesis (H_1). The hypothesis you would believe if H_0 were rejected. Usually states that an effect exists.

Asymptotic. Pertaining to behaviour as $n \to \infty$. The Central Limit Theorem is an asymptotic result. MLE is asymptotically efficient.

Base rate. The prior probability of an event in the population. The base rate fallacy is ignoring this when interpreting conditional probabilities.

Bayes' theorem. The formula relating posterior, likelihood, prior, and evidence. Derived from the symmetry of the multiplication rule.

Bayesian. An approach that treats parameters as random variables with probability distributions. Contrast with *frequentist*.

Bessel's correction. Dividing by $n - 1$ instead of n when estimating sample variance, to get an unbiased estimator.

Beta (β). Two meanings depending on context: (1) a coefficient in a regression model; (2) the Type II error rate. Power is $1 - \beta$ in the second sense.

Bias (of an estimator). The difference between the estimator's expected value and the true parameter value. An estimator is unbiased if its expected value equals the parameter.

Bonferroni correction. A multiple-testing adjustment that divides α by the number of tests. Conservative.

Bootstrap. A resampling technique introduced by Efron (1979). Treat the empirical distribution as a stand-in for the population, draw samples with replacement, and use the resulting statistics as a surrogate for the true sampling distribution.

Central Limit Theorem (CLT). The sum (or mean) of many independent random variables with finite variance is approximately Normally distributed regardless of the original distribution.

Confidence interval (CI). A random interval that contains the true parameter with a specified probability *under repeated sampling*. Not a Bayesian credible interval.

Conjugate prior. A prior distribution that, combined with a given likelihood, yields a posterior in the same family. Beta is conjugate to Binomial; Normal is conjugate to Normal (known variance).

Counterfactual. What would have happened under a different treatment. Never directly observed.

CUPED. Controlled experiment using pre-experiment data. A variance- reduction technique for A/B tests that regresses the outcome on a pre-experiment covariate.

Degrees of freedom (df). The number of independent values that can vary in a statistic. For the sample variance, `df = n - 1`.

Difference-in-Differences (DiD). A causal inference method that compares pre-post changes in a treated group to those in a control group, assuming parallel trends.

Effect size. The magnitude of a difference or relationship. Statistical significance speaks to whether an effect is real; effect size speaks to how big.

Expected value. The probability-weighted average of a random variable's possible values.

F-statistic. A ratio of variances used in ANOVA and in the overall regression F-test.

Family-wise error rate (FWER). The probability of making at least one Type I error across a family of tests. Controlled by Bonferroni.

False discovery rate (FDR). The expected proportion of false discoveries among rejections. Controlled by Benjamini-Hochberg.

Fisher information. A measure of how much information a sample provides about a parameter. Defined as the expected negative second derivative of the log-likelihood.

Frequentist. An approach that treats parameters as fixed unknowns and interprets probability as long-run frequency.

Hypothesis. A statement about a parameter or distribution. The null (H_0) is the default; the alternative (H_1) is what you hope to show.

Independence. Two events are independent if knowledge of one does not change the probability of the other.

Law of Large Numbers (LLN). As $n \to \infty$, the sample mean converges to the population mean.

Likelihood. The probability of the observed data viewed as a function of the parameter.

Log-likelihood. The natural logarithm of the likelihood. Easier to differentiate and numerically stable.

MAP (Maximum a Posteriori). The parameter value that maximises the posterior distribution. Equivalent to MLE plus a prior.

Marginal likelihood. `P(data)` under Bayes' theorem, obtained by integrating out the parameter.

Maximum likelihood estimation (MLE). The parameter value that maximises the likelihood of the observed data.

Minimum detectable effect (MDE). The smallest true effect that an experiment can reliably detect with a given sample size, α, and power.

Multiple testing problem. The inflation of false-positive rates when performing many hypothesis tests simultaneously.

Null hypothesis (H_0). The default hypothesis, typically "no effect."

Odds. `p / (1 - p)` where `p` is a probability. The odds of an event with probability 0.8 are 4.

Odds ratio. The ratio of odds between two conditions. In logistic regression, `e^β` is the odds ratio associated with a one-unit change in the corresponding predictor.

OLS (Ordinary Least Squares). The linear regression estimator that minimises the sum of squared residuals.

Parallel trends. The critical assumption behind DiD: in the absence of treatment, the treated and control groups would have evolved the same way.

Permutation test. A non-parametric test that computes a p-value by reshuffling labels many times and counting how often the resulting test statistic exceeds the observed one.

Population. The complete set of entities you care about.

Posterior distribution. The updated distribution of a parameter after observing data, computed via Bayes' theorem.

Power ($1 - \beta$). The probability of correctly rejecting H_0 when H_1 is true. Depends on sample size, effect size, variance, and α.

Prior distribution. Your belief about a parameter before observing data.

p-value. The probability of observing data at least as extreme as yours, assuming H_0 is true. Not the probability that H_0 is true.

R-squared (R^2). The proportion of variance in the outcome explained by a regression model.

Random variable. A function mapping outcomes of a random experiment to numbers.

Resampling. Drawing new samples from an existing dataset, with or without replacement.

Residual. The observed value minus the predicted value in a regression model.

Sample. A subset of the population actually observed.

Sample ratio mismatch (SRM). A discrepancy between the expected and observed allocation to experiment arms. Indicates pipeline bugs.

Sampling distribution. The distribution of a statistic across hypothetical repeated samples.

Sigmoid. The logistic function `σ(z) = 1 / (1 + e^(-z))`.

Significance level. Synonym for α.

Standard deviation. The square root of the variance.

Standard error (SE). The standard deviation of a sampling distribution; a measure of how precise an estimate is.

Statistic. Any function of the sample (e.g. mean, variance, regression coefficient). Random because the sample is.

Sufficient statistic. A statistic that captures all the information in the data relevant to a parameter.

t-distribution. The distribution of `(x̄ − μ) / (s / √n)` under Normal data with `n − 1` degrees of freedom. Converges to standard Normal as `n → ∞`.

t-statistic. A standardised measure of how far a sample estimate is from a hypothesised value, expressed in standard errors.

Type I error. Rejecting H_0 when it is true. Rate is α.

Type II error. Failing to reject H_0 when H_1 is true. Rate is β.

Variance. The expected squared deviation from the mean.

Appendix C — Common Mistakes Checklist

A short list of the mistakes most commonly made when applying Module 2 tools. Skim this before submitting any analysis.

1. **Confusing `P(A | B)` with `P(B | A)`.** Always read the order of conditioning carefully. The prosecutor's fallacy is exactly this confusion.
2. **Ignoring the base rate.** A "99% accurate" test on a 0.5%- prevalence condition has high false-positive rate. Always plug into Bayes.
3. **Saying "the 95% CI has a 95% probability of containing the parameter."** It doesn't; that's the Bayesian interpretation. Use a credible interval if you want that statement.
4. **Treating a p-value as `P(H₀ | data)`.** p is `P(data ≥ observed | H₀)`. Never the reverse.
5. **Running an underpowered experiment and concluding "no effect."** A non-significant p-value with low power is uninformative. Always compute power up front.
6. **Testing many hypotheses without correction.** Testing 20 features at `α = 0.05` expects one false positive by chance. Apply Bonferroni or BH-FDR.
7. **Skipping the SRM check.** Before interpreting any A/B test result, confirm the allocation ratio is as expected.
8. **Interpreting `R²` without checking assumptions.** High `R²` on a misspecified model is worthless. Plot residuals.
9. **Omitting the dummy-variable base.** Either drop one dummy or fit without an intercept — never both all `k` dummies *and* an intercept, which produces a singular `XᵀX`.
10. **Confusing ANOVA with post-hoc tests.** ANOVA says "somewhere there's a difference." Use Tukey HSD to find which pairs differ.
11. **Interpreting a logistic regression coefficient as a linear effect.** It's a log-odds effect. Exponentiate to get the odds ratio.
12. **Applying CUPED with a covariate measured during the experiment.** The covariate must be pre-experiment, full stop.
13. **Trusting DiD without a parallel-trends check.** Always plot the pre-trends. Run a placebo test. DiD without this check is just a pair of averages with extra steps.
14. **Using `n` instead of `n − 1` for sample variance.** Bessel's correction exists for a reason. NumPy defaults to `ddof=0` (population); pass `ddof=1` for sample.
15. **Reporting p-values without effect sizes and CIs.** "p = 0.03" alone tells a reader nothing about magnitude or precision. Always pair it with a point estimate and a confidence interval.
16. **Treating aggregated data as transaction-level.** Daily totals hide heterogeneity. Model at the finest level you can and aggregate only for reporting.
17. **Ignoring temporal structure in the bootstrap.** Standard bootstrap assumes i.i.d. For time series, use a block bootstrap.

18. **Using the wrong population for the base rate.** "Disease prevalence is 0.1%" — in whom? Hospitalised patients have a very different base rate from the general public.
19. **Not logging the experiment design.** If you didn't log the hypothesis, power analysis, and SRM threshold *before* looking at results, you can't defend against post-hoc rationalisation.
20. **Assuming correlation implies causation.** It doesn't. Ever. Causal claims require either randomisation or explicit causal assumptions (DAGs, parallel trends, unconfoundedness).

Appendix D — Quick Decision Table

Use this table to decide which tool from Module 2 fits a given question.

Question	Tool
"Given a positive signal, what's the probability of X?"	Bayes' theorem (Lesson 2.1)
"What's my best guess at the parameter?"	MLE (Lesson 2.2)
"What's my best guess with prior knowledge?"	MAP (Lesson 2.2)
"Is the sample mean reliably different from zero?"	One-sample t-test (Lesson 2.3)
"Are two groups different on average?"	Two-sample t-test or permutation test
"What's a distribution-free CI for a weird statistic?"	Bootstrap percentile or BCa
"Is my randomisation working?"	SRM chi-squared check (Lesson 2.4)
"Do I have enough sample size?"	Power analysis (Lesson 2.3/2.4)
"How much does X drive Y, controlling for Z?"	Linear regression with covariates (2.5)
"Is this coefficient statistically significant?"	t-statistic on $\hat{\beta}$ (Lesson 2.5)
"Does my model do better than random?"	F-test (Lesson 2.5)
"How much variance am I explaining?"	R^2 / adjusted R^2 (Lesson 2.5)
"Will the customer click?"	Logistic regression (Lesson 2.6)
"How much more likely is Y given a 1-unit change in X?"	Exponentiated logistic coefficient (2.6)
"Do three or more groups have different means?"	ANOVA + Tukey HSD (Lesson 2.6)
"How do I detect smaller effects with the same sample?"	CUPED (Lesson 2.7)
"What's the effect of a policy I can't randomise?"	DiD with parallel-trends check (2.7)
"How do I tell a story with all of the above?"	Capstone pipeline (Lesson 2.8)

Appendix E — Further Reading

Books that go deeper into the Module 2 material, roughly in order of difficulty:

1. **OpenIntro Statistics** (Diez, Barr, Çetinkaya-Rundel). Free online. The best gentle introduction; covers everything in Lessons 2.1–2.5 with clear writing.
2. **Statistical Rethinking** (McElreath). A deeply Bayesian introduction with excellent intuition and a hands-on computational style. Excellent companion to Lessons 2.1 and 2.2.
3. **All of Statistics** (Wasserman). A fast-paced, mathematically rigorous survey. Every chapter has something to teach a working data scientist.
4. **Mostly Harmless Econometrics** (Angrist, Pischke). The best practical guide to causal inference from observational data. Core reference for Lesson 2.7.
5. **Trustworthy Online Controlled Experiments** (Kohavi, Tang, Xu). Practical guide to A/B testing at scale, including CUPED, sequential testing, and interference. Written by the team that ran thousands of experiments at Microsoft.

6. **Causal Inference: The Mixtape** (Cunningham). Free online. Friendly introduction to DiD, IV, RDD, and synthetic control.
7. **The Elements of Statistical Learning** (Hastie, Tibshirani, Friedman). Free online. The bridge from classical statistics to machine learning; will be your companion through Module 3.

Papers worth reading:

- Efron (1979), "Bootstrap Methods: Another Look at the Jackknife." The paper that invented the bootstrap.
- Deng, Xu, Kohavi, Walker (2013), "Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data." The original CUPED paper.
- Benjamini & Hochberg (1995), "Controlling the False Discovery Rate." The paper that gave us BH-FDR.
- Rubin (1974), "Estimating Causal Effects of Treatments in Randomized and Nonrandomized Studies." Foundation of potential outcomes.

Appendix F — Self-Test: 20 Questions for Module 2

Use this as a final check before moving to Module 3. If you can answer all 20 without looking, you're ready.

1. State Bayes' theorem and name each of the four components.
2. What is the difference between `P(A | B)` and `P(B | A)`? Give an example where they differ dramatically.
3. What does it mean for two events to be independent? Give a formula.
4. Define expected value and give an example.
5. Derive the two equivalent forms of variance.
6. Why does sample variance divide by `n - 1` rather than `n`?
7. State the Central Limit Theorem in one sentence.
8. What is the *correct* interpretation of a 95% confidence interval? What is the *incorrect* one that most people use?
9. Derive the MLE for the Normal mean and variance.
10. Explain the p-value in plain language. What is it *not*?
11. What is power, and what four things determine it?
12. Describe how a permutation test works, step by step.
13. What is an SRM, and how do you detect it?
14. Derive the OLS normal equations: $\hat{\beta} = (X^T X)^{-1} X^T y$.
15. What does a regression coefficient's t-statistic test, and what's its sampling distribution under the null?
16. Define R^2 and explain what it measures.
17. What is the sigmoid function, and why is it used in logistic regression?
18. How do you interpret e^{β} from a logistic regression?
19. Derive the CUPED variance-reduction formula $Var(Y_{adj}) = Var(Y)(1 - \rho^2)$.
20. State the parallel-trends assumption for Difference-in- Differences. How do you test it?

Solutions are throughout Lessons 2.1–2.7. No cheat sheet.

End of Module 2 textbook chapter. Continue to Module 3 — Supervised Machine Learning: Theory to Production.